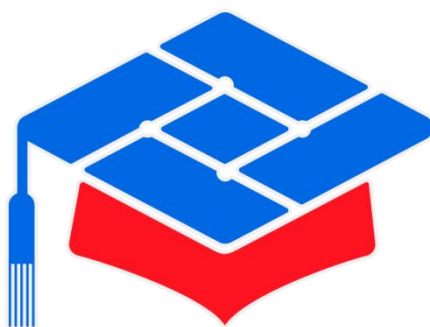




第十届

全国大学生集成电路创新创业大赛

报告类型：_____ 技术文档 _____

参赛杯赛：_____ “紫光同创” 企业命题 _____

作品名称：_____ 基于 RK3568 MES2L100H 的智能交通感知系统 _____

队伍编号：_____ CICC1002220 _____

团队名称：_____ 能跑就能用 _____

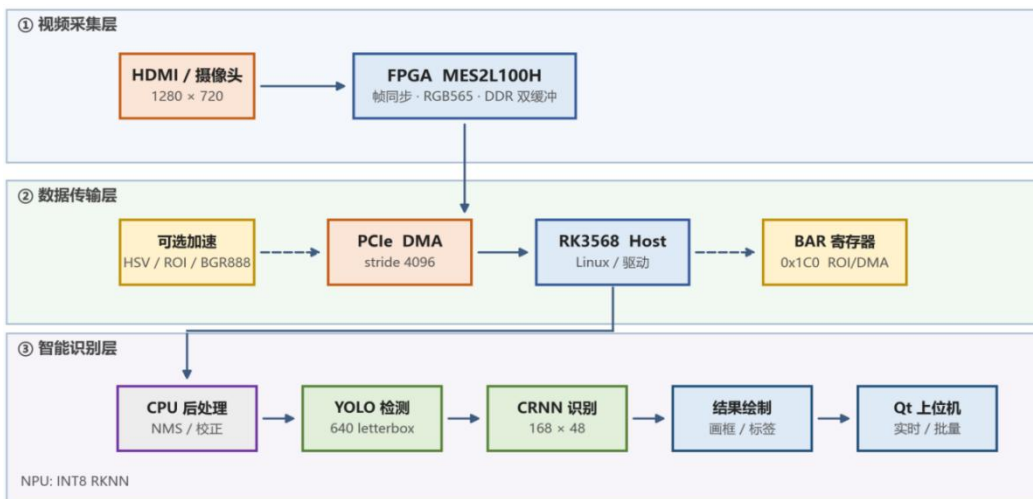
快速预览简介

1、作品概述

本作品采用 FPGA 采集传输 + NPU 识别的异构架构，从硬件层面打通“采集—传输—推理—显示”全链路。紫光 MES2L100H 输出 1280×720@RGB565 (stride 4096)，DDR 双缓冲后经 PCIe DMA 映射至 RK3568；板端 INT8 RKNN 部署 YOLO (640 letterbox 车牌定位) + CRNN (168×48 字符识别)，NPU 承担主推理，CPU 负责 NMS/校正。

Qt 支持 PCIe 实时预览与文件夹图片或视频批量处理，分段计时便于优化延迟。实测验收测试集 (CCPD 分层 + 自采共 500 张)：核心算法延迟 process_wall≈65 ms、整帧延迟≈98 ms，车牌定位的 mAP0.5 达 98.2%、字符识别的 mAP0.5 达 98.1%。

系统框体如下图所示：



2、作品完成情况

- ◆ 已完成视频的采集输入、识别、输出、显示的**整个链路**。
- ◆ **标记准确率可达 98.2%**：针对不同场景，对一个或者多个车牌进行标记。
- ◆ **识别准确率可达 98.1%**：不同角度及光照强度下。
- ◆ **核心算法处理延迟 65 ms、整帧延迟 98ms。**
- ◆ **特殊车牌识别**：大黄牌、小黄牌、领馆车牌、使馆车牌。

3、创新点

◆ FPGA 与 RK3568 异构分工，数据面与算力面解耦

前端紫光 MES2L100H 承担 HDMI 采集、RGB565 帧同步、DDR 双缓冲与 PCIe DMA 传输，把视频 IO 与带宽压力留在可编程逻辑侧；RK3568 NPU 专注 YOLO+CRNN 语义识别，CPU 负责解码与业务交互。软硬职责边界清晰，采传链路 with 算法迭代可独立优化，便于现场部署与后续扩展。

◆ YOLO+CRNN 统一深度学习主路径，INT8 板端部署与分段计时闭环优化

检测与识别均采用 INT8 RKNN 板端推理，PC 端 ONNX 与板端模型同源对齐，Qt 实时预览与 CCPD 批量评测共用同一套核心链路。引入分段计时，定位 PCIe 取图、NPU 推理、解码绘制等瓶颈，经 FP32→INT8→绘制精简→链路整合四轮优化，使核心算法延迟 process_wall 由约 105 ms 降至 65 ms，精度仍保持 98% 以上。

◆ 全链路板端闭环 + 可复现评测体系

打通 FPGA 采集、PCIe 传输、RK3568 推理与 Qt 展示的完整硬件闭环，而非 PC 端离线 demo。建立 CCPD 分层抽样与自采难例共 500 张（五类场景各 100 张）的统一验收评测流程，Qt 批量评测与 PCIe 实时流共用同一 RKNN 链路，精度与延迟均可分段计时、可复现验证。

4、工程优化

采用 [latency] 分段计时定位瓶颈，经四轮工程优化将 process_wall 从 V0 基线 FP32 的 105 ms 逐步降至 V3 的 65 ms，V3 通过 decode 早筛与链路整合进一步收敛。整体延迟降低约 38%，mAP 98.2%、识别 98.1% 保持不变。



目录

第十届.....	- 1 -
全国大学生集成电路创新创业大赛	- 1 -
快速预览简介	- 2 -
1、作品概述	- 2 -
2、作品完成情况	- 2 -
3、创新点	- 3 -
4、工程优化	- 3 -
第一章 系统总体设计方案	- 6 -
1.1 系统总体架构	- 6 -
1.2 系统工作流程	- 7 -
1.3 关键技术路线	- 8 -
第二章 硬件架构设计	- 9 -
2.1 总体硬件架构	- 9 -
2.2 存储与外设接口	- 10 -
2.3 时钟、电源	- 10 -
第三章 RK3568 端算法训练、部署与性能调优	- 12 -
3.1 算法方案与模型结构	- 12 -
3.2 模型训练与数据	- 14 -
3.3 软件架构与推理流程	- 16 -
3.4 模型部署与系统接入	- 18 -
3.5 性能调优与分析	- 19 -
第四章 FPGA 子系统技术文档	- 21 -
4.1 概述	- 21 -
4.2 设计框图	- 21 -
4.3 功能介绍	- 22 -
4.4 HDMI 视频采集模块	- 23 -
4.5 DDR 帧缓存模块	- 24 -
4.6 PCIe 传输模块	- 26 -

4.7	模块仿真	- 28 -
4.7.1	HDMI 视频采集模块仿真	- 28 -
4.7.2	DDR 帧缓存模块仿真	- 30 -
4.7.3	DMA 协议栈仿真	- 32 -
4.8	静态时序分析	- 33 -
4.9	FPGA 资源占用分析	- 35 -
第五章	测试报告	- 36 -
5.1	测试流程与环境	- 36 -
5.2	离线精度测试	- 37 -
5.3	板端功能与 PCIe 实时测试	- 37 -
第六章	核心代码注释	- 39 -
6.1	FPGA 端核心代码 (Verilog)	- 39 -
	Sobel 边缘检测模块	- 39 -
	PCIe DMA 控制器	- 41 -
6.2	RK 端核心代码 (C++)	- 43 -
	ROI 精细化定位与几何校正	- 43 -
	LPR 模型推理调用	- 45 -

第一章 系统总体设计方案

1.1 系统总体架构

本系统采用「FPGA 实时采集与预处理 + RK（瑞芯微）应用处理」的分层架构。FPGA 作为 PCIe Endpoint（设备端），负责 CMOS 接口、图像缓存与面向主机的 DMA 数据打包；RK 芯片作为 PCIe Root Complex 及主控 SoC，负责枚举外设、接收帧数据、解析元数据并运行显示、存储、网络与智能算法等业务。

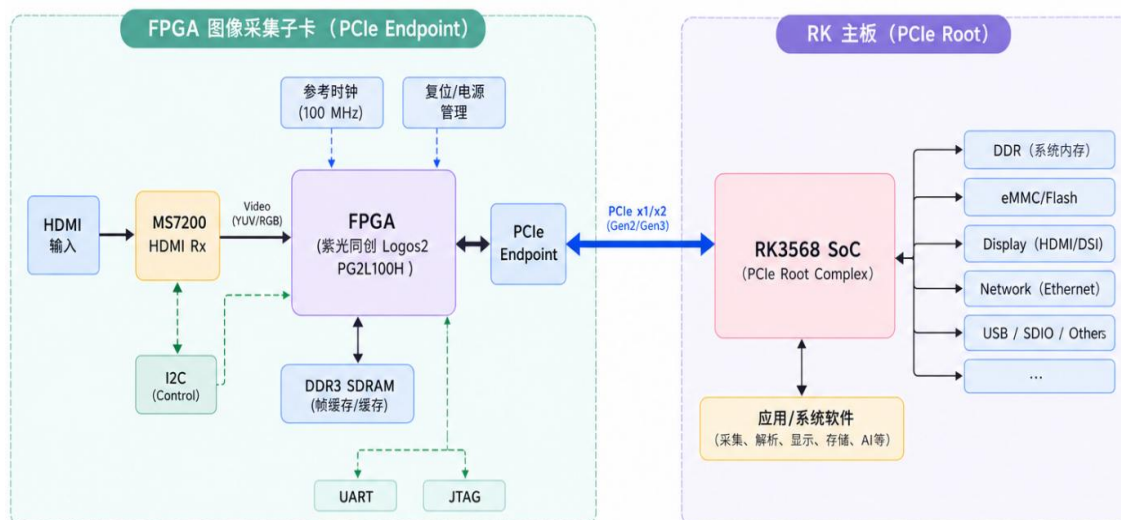


图 1-1 系统总体架构框图

物理与逻辑结构：

板上，CMOS 传感器通过并行 DVP 及 I2C 连接 FPGA；FPGA 外挂 DDR3，用于整帧图像的行/帧缓存与读写仲裁；FPGA 与 RK 之间通过标准 PCIe 链路连接，由 RK 侧驱动完成 BAR 映射与 DMA 缓冲管理。

数据流（主路径）：

像素数据在 FPGA 内完成 RGB888→RGB565 整形后写入 DDR3 帧缓存，在主机发起 DMA 时按节拍读出，经 PCIe 连续上传至 RK3568；板端在 RGB565→RGB888 解码后送入统一的 YOLO+CRNN 流水线，先由 YOLO 完成车牌定位与四角点回归，再经透视校正裁剪送入 CRNN 完成字符与颜色识别。为便于主机对齐，逻辑在图像数据前拼接固定长度的轻量帧头（携带 magic、分辨率、帧号），保证 RK 侧一次读操作即可得到完整帧。

软硬件分工：

FPGA：HDMI 接收芯片配置与时序对齐、RGB888→RGB565 格式转换、DDR3 帧缓存与四缓冲乒乓调度、PCIe/DMA 数据打包与帧边界对齐。

RK：PCIe 枚举与驱动、整帧 DMA 接收与帧头解析、RGB565→RGB888 解码、YOLO 车牌检测 + CRNN 字符识别（NPU 加速）、透视校正与 NMS（CPU）、Qt 显示与 (frame_id, plate_text, plate_type, conf) 结果输出。

1.2 系统工作流程

（1）上电与初始化阶段

系统上电后，FPGA 与 RK 并行启动。FPGA 侧完成板级复位释放、PLL/时钟锁定、DDR3 控制器初始化；待 DDR 初始化完成并同步到相机控制时钟域后，依次对 CMOS 传感器执行上电时序与 I2C 寄存器配置，得到稳定的像素时钟、场同步、行有效与 8bit 数据。RK 侧完成 Boot、PCIe 子系统枚举，识别 FPGA 设备，映射 BAR、分配 DMA 缓冲区并完成驱动内参数（如单帧长度、缓冲个数）与应用的对接。

（2）稳态采集与缓存阶段

HDMI 源按 1280×720@RGB565 输出视频流。FPGA 将 24 位 RGB888 按 R5G6B5 截位转换为 RGB565 后，经写 FIFO 跨时钟域写入 DDR3 帧缓存；四缓冲乒乓调度使连续采集与 DMA 上传可并行进行，互不覆盖。

（3）主机取数与帧边界阶段

当 RK 应用或驱动发起一次 PCIe DMA 读（或等价的块传输）时，FPGA 在 DMA 写数据请求有效期间，按固定节拍输出 先 metadata、后图像 的连续字节流：帧首固定长度（如 224 字节）承载 magic、宽高、帧号、ROI 数量与 ROI 列表等；其后为 1280×720×2 字节的 RGB565 原图数据。帧与帧之间由 FPGA 内 beat 计数与帧结束/长空闲对齐逻辑保证 新一帧总是从 metadata 起点开始，避免主机侧 magic 漂移或 burst 间误对齐。

（4）RK 侧处理阶段

RK 每收到一帧完整缓冲，先校验帧头并得到原图起始指针，再完成 RGB565→RGB888 转换；随后送入 YOLO 检测得到车牌框与四角点，经透视校

正裁剪并缩放为 168×48 送入 CRNN, CTC 解码得到字符序列与底色类别；最后在原图上叠加画框与识别结果，通过 Qt 显示或本地日志输出。若帧头解析失败，应用丢弃本帧并记录日志。

1.3 关键技术路线

（1）系统分工与缓存架构：

整体采用 FPGA + RK 异构架构。FPGA 承担传感器接口、帧同步、DDR 仲裁、PCIe 打包等硬实时高吞吐任务；RK 负责驱动、缓冲队列、解码、算法与网络协议。片外 DDR3 作为帧缓存中枢，用于缓解传感器、PCIe 与主机调度间的速率不匹配，并为多路输入和更高分辨率扩展预留空间。

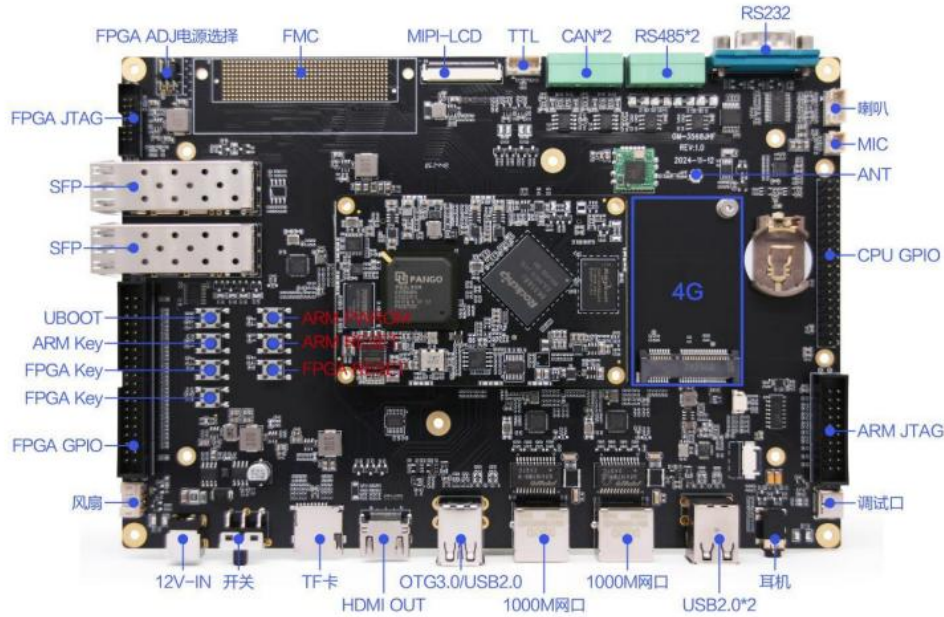
（2）数据传输与完整性保障：

PCIe 采用单 DMA 通道输出“metadata + 图像数据”的连续帧缓冲，降低 RK 侧异步描述符管理复杂度。FPGA 需通过 beat 计数、帧尾检测、空闲对齐和重启同步，确保每帧 metadata 唯一完整，并与图像字节数严格对应。跨时钟域需进行同步、锁存和 STA 收敛，避免亚稳与错帧。

（3）图像处理与软件路线：

资源允许时，将灰度、滤波、边缘、二值、形态学、投影与 ROI 粗筛等高吞吐、低参数处理放入 FPGA，输出候选 ROI；颜色语义、复杂分类和大模型推理留在 RK。RK 软件统一帧长度常量，封装帧解析接口，校验 magic、拆分 meta 与 rgb565，并通过 frame_id 与 ROI 坐标完成联调对齐。

第二章 硬件架构设计



2.1 总体硬件架构

系统硬件由 RK3568 SoC 主板（PCIe Root）与 FPGA 图像采集子卡（PCIe Endpoint）构成，二者经标准 PCIe 互联。子卡侧以紫光同创 Logos2 系列 PG2L100H(模块型号 MES2L100H)为核心，外围包括 HDMI 接收芯片(MS7200)、DDR3 SDRAM、PCIe 差分对与参考时钟输入，并预留 UART、JTAG 等接口用于寄存器调试与现场排障。主数据路径为：HDMI 视频经 MS7200 进入 FPGA，经整形与帧缓存写入 DDR3；在主机 DMA 调度下，FPGA 将帧头与图像数据连续送至 PCIe 链路，由 RK 侧接收与解析。辅助路径为 I2C 对 MS7200 寄存器配置、板级复位与电源上电顺序控制。

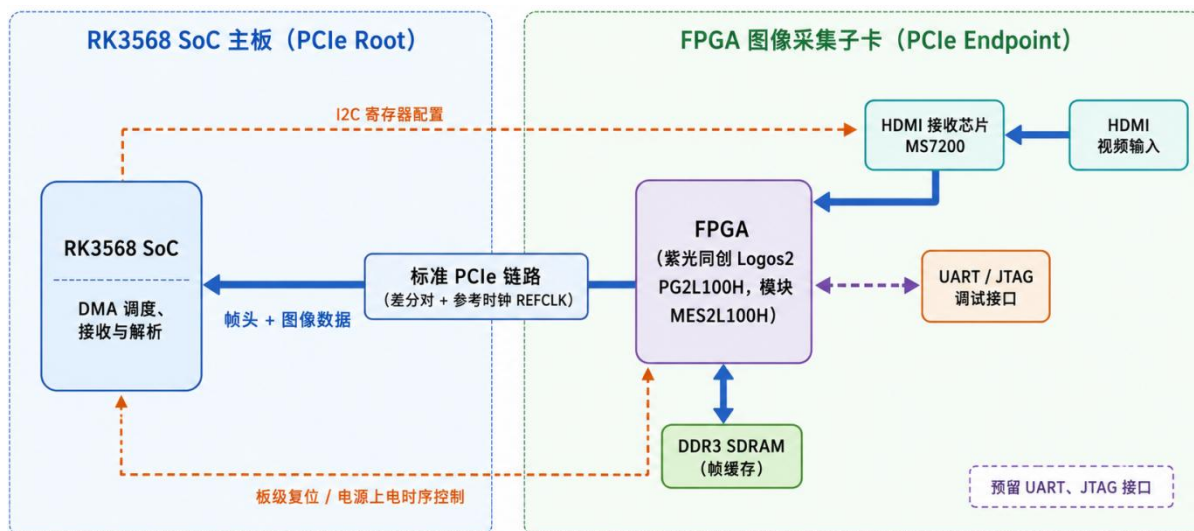


图 2-1 硬件总体框图

2.2 存储与外设接口

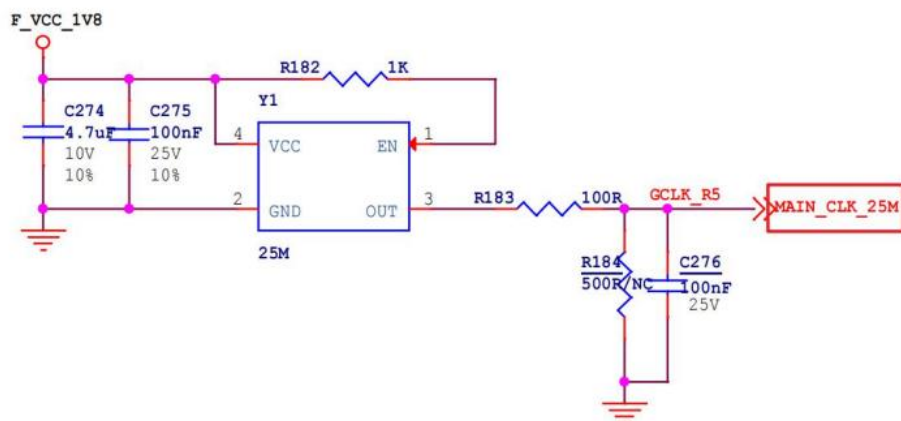
存储 (DDR)：DDR3 与 FPGA 专用 IO bank 相连，包含 地址、控制、数据、DQS/DM 等信号组；布局布线遵循 等长、阻抗、同层参考面 等规则，保证 PHY 训练与长期稳定性。DDR 在系统中主要作为 帧缓存，缓释传感器连续输出与 PCIe 突发传输之间的速率差异。

外设与接口：

- (1) 图像传感器：并行数据、场/行同步、像素时钟、I2C、复位与供电；
- (2) PCIe：TX/RX 差分对、参考时钟、复位；保持 阻抗连续、过孔与 stub 最小化，满足链路预算。
- (3) 调试与配置：UART 至 FPGA 内 APB/寄存器访问；JTAG 用于下载比特流与在线调试（按生产流程决定是否预留测试点）。

2.3 时钟、电源

时钟：下图为 25MHz 有源晶振电路，此时钟连接至 FPGA 的全局时钟管脚上，可为 FPGA 提供输入参考时钟。



核心板 25MHz 有源晶振电路

传感器 PCLK 由模组输出，在 FPGA 内作为异步像素域；FPGA 内部由 PLL/MMCM 产生 DDR 控制器时钟、PCIe 用户时钟、配置用低速时钟等。复位包括板上电复位、PCIe PERST，保证 DDR 初始化完成后再释放与像素相关的关键逻辑。跨时钟域数据在帧/场边界或握手后传递，避免亚稳与错帧。

电源：为 FPGA、DDR、传感器、电平转换等分配独立电源轨，核对上电顺序与电流峰值。

第三章 RK3568 端算法训练、部署与调优

本章介绍本队在 RK3568 平台上实现车牌检测与识别的完整技术路径。系统采用“FPGA 轻量预处理 + RK3568 深度学习”的分工架构：FPGA 是可编程硬件芯片，负责高速采集视频帧并通过 PCIe 传输至板端；RK3568 是瑞芯微提供的嵌入式 AI 平台，内置 NPU 负责车牌检测与识别推理；Qt 上位机实现实时显示与结果可视化。本章重点阐述算法设计思路、数据与训练策略、板端部署方案，以及通过多轮优化所实现的精度与时延提升效果。

从工程闭环角度看，本章内容覆盖“模型选型—数据构建—训练迭代—格式转换—板端部署—性能剖析”六大环节。各环节均设置可量化指标：检测端以 $mAP@0.5$ 衡量定位质量，识别端以整牌准确率衡量字符与颜色输出质量，部署端以 `process_wall`(算法处理延迟)与整帧周期衡量实时性。最终在保证检测 mAP 与识别准确率均不低于 98% 的前提下，将算法处理延迟优化至约 65 ms，满足路口视频实时演示与联调需求。

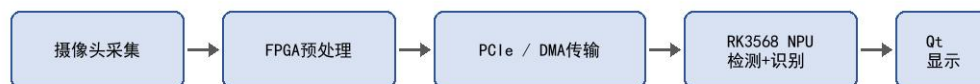


图 3-1 系统总体数据流

如图 3-1 所示，视频数据自摄像头进入后，首先由 FPGA 完成采集同步与格式整理，再经 PCIe / DMA 通道送入 RK3568。板端完成检测与识别后，结果回写 Qt 界面。该链路将“高带宽采图”与“智能推理”解耦，既保证帧率，又便于算法独立迭代。

3.1 算法方案与模型结构

车牌识别在工程上通常分为“检测”和“识别”两个子任务。检测的目标是在整张图片里找到车牌在哪里，并给出框的位置和大小；识别的目标是把已经裁好的车牌图片读成文字（如“京 A12345”）和颜色（蓝牌、黄牌等）。本项目采用两阶段方案：先检测再识别。这样可以先在大图上定位车牌，再对小区域做精确识别，

比“一步到位”更容易做好。

之所以坚持两阶段而非端到端一体模型，主要基于三点工程考量。其一，检测与识别的输入尺度差异显著：检测需要在 1280×720 全景图上工作，识别只需关注 168×48 的牌面区域；其二，两阶段便于分别训练与替换：检测模型升级时不必重训识别网络，识别字符集扩展时也不必重训检测器；其三，板端调试更清晰——检测失败与识别失败可以在日志中明确区分，缩短联调周期。实践表明，该方案在精度、可维护性与部署灵活性之间取得了较好平衡。

检测模型基于改进的轻量 YOLOv5 网络（yolov5n-0.5）。YOLO 是一种常用的目标检测算法，能在一次前向传播中同时预测多个目标的位置和类别。本项目在标准 YOLO 基础上增加了车牌四角关键点回归，用于后续把拍斜的车牌矫正。输入图像先经过 letterbox 处理缩放到 640×640。letterbox 的意思是按原图长宽比等比缩放，不足的部分用灰色填充，避免图像被拉伸变形。预定尺寸锚框（anchor）是在特征图上预先设置的参考框模板，网络只需预测目标相对锚框的偏移量，从而加快学习和推理速度。

检测头设计为三层特征图网络（P3、P4、P5），分别对应 8、16、32 像素的感受野（stride）。这样可以同时检测大、中、小不同尺寸的车牌。每层配 3 个预定尺寸的锚框（anchor）作为参考模板。类别分为单层牌和双层牌两种。推理后还要经过 NMS（非极大值抑制）去除重复框，只保留最可靠的检测结果。NMS 的基本思想是：当多个框高度重叠时，只保留置信度最高的那个，其余框按交并比阈值抑制，从而避免同一车牌被重复框选，保证后续识别输入干净、唯一。

识别模型采用 CRNN 结构（myNet_ocr_color）。CRNN 适合处理字符串识别，可以把车牌上的字符逐个读出来。其典型组成包括：卷积骨干提取局部笔画特征，循环网络建模字符序列依赖，以及基于 CTC 的序列解码层。输入固定为 168×48 像素的车牌图像，输出为字符序列和颜色分类。字符解码采用 CTC 算法，能处理字符长度不固定的情况。CTC（Connectionist Temporal Classification）无需对每个字符做严格的逐帧对齐标注，适合车牌这类长度可变、字符间距不稳定的序列任务。系统支持蓝牌、黄牌、新能源、警用、双层牌、使领馆等多种常见中文车牌格式。

表 3-1 检测与识别模型概要对齐

维度	检测模型	识别模型
网络结构	YOLOv5n-0.5 + 关键点	CRNN + 颜色分支
输入尺寸	640×640（letterbox）	168×48（透视矫正后）
输出内容	框 + 4 角点 + 单/双层	号牌字符 + 5 色分类
训练框架	PyTorch	PyTorch
板端部署	plate_detect_int8.rknn	plate_rec_color_int8.rknn
硬件加速	RK3568 NPU	RK3568 NPU

表 3-1 从结构、输入、输出、训练与部署五个维度给出了对齐说明。可以看出，检测与识别在板端均以 INT8 RKNN 形式驻留 NPU，二者通过统一的推理库串联，形成“定位—矫正—读码—着色”的完整闭环。

3.2 模型训练与数据

数据是深度学习模型的基础。检测训练以 CCPD2019 公开数据集为核心，该数据集包含超过 30 万张中文车牌图片，覆盖清晰、模糊、倾斜、夜间、难例等多种场景。在此基础上，本队融合了 CRPD 数据和自采数据，增强了本地路口视角和实际部署环境的适应性。识别模型使用独立的车牌小图数据集单独训练，检测与识别分离迭代，便于分别优化。

自采数据主要来自项目联调路口与演示场景，采集时兼顾白天/黄昏、正拍/斜拍、近距/远距等条件，并对双层牌、新能源牌等相对少见样本进行了定向补充。标注方面，检测任务标注外接框与四角点，识别任务标注字符序列与颜色类别。全部样本经分层抽样划入训练/验证/测试子集，保证难例子集在各划分中比例一致，避免“训练集简单、测试集简单”导致的虚高指标。

训练过程采用逐步优化策略。第一阶段使用 CCPD 基础数据集进行预训练，建立初始检测能力；第二阶段融入 blur、tilt、challenge 等难例子集，配合 mosaic、mixup 数据增强。mosaic 是把多张图片拼成一张进行训练，mixup 是将两张图片按比例混合，二者都能增加样本多样性，提升模型对复杂场景的鲁棒性；第三阶段加入自采数据做场景微调，使模型更好适应本项目的摄像角度和光照条件。识别模型同样经过“通用数据预训练 → 难例增强 → 自采数据微调 → 后处理与部署对齐”四步迭代。每一轮训练后均在同一套分层抽样测试集上评测，确保指标可横向对比。

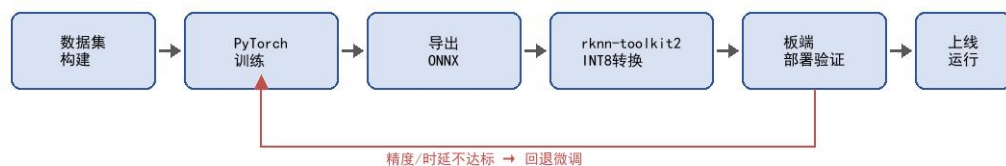


图 3-2 模型训练、转换与部署闭环

图 3-2 给出了从数据构建到上线运行的闭环。当板端验证发现精度或时延不达标时，可回退到微调或量化校准环节继续迭代。该闭环保证了“离线指标—板端指标—演示效果”三者可对照、可追溯。

训练超参数方面，检测模型采用 SGD 优化器，初始学习率 $1e-2$ ，配合余弦退火与预热；输入分辨率 640，批量大小按显存容量取 16~32；训练轮次按阶段递进，预训练约 100 epoch，难例增强约 60 epoch，场景微调约 40 epoch。识别模型以 Adam 为主，学习率 $1e-3$ ，输入固定 168×48 ，并在字符类不平衡时采用加权采样，提升少见省份简称与特殊字符的召回。上述配置均在 PC 工作站完成，随后导出 ONNX 再转入板端转换。

训练完成后，模型先导出为 ONNX 格式，再用 rknn-toolkit2 转换为 RK3568 可执行的 RKNN 格式。ONNX 是通用的深度学习模型交换格式，便于在 PC 上验证和跨平台部署；RKNN 是瑞芯微提供的板端推理格式，能把通用模型压缩并部署到 NPU 上加速运行。NPU（神经处理单元）是专门用于深度学习计算的硬件加速器，相比 CPU 推理速度更快、功耗更低。转换时采用 INT8 量化（将浮点权重压缩为 8 位整数），在精度基本保持的同时显著提升推理速度。本队在板端和 PC 端分别验证了 ONNX 与 RKNN 的结果一致性，确保离线评测与实际部署结果可对照。

量化环节特别强调校准集的代表性。校准图像从训练分布中分层抽取，覆盖清晰牌、模糊牌、夜间牌与斜拍牌，避免校准集过于“干净”导致量化后难例精度下降。转换完成后，在板端用验收测试集（CCPD 分层抽样 + 自采难例共 500 张）复测 mAP 与识别准确率，并与 PC 端 ONNX 结果逐项比对；差异控制在 0.5 个百分点以内后，才进入正式部署。

本队将精度提升视为一条可复现的工程链路：先通过数据扩充打通“能检测到”

的上限，再用自采数据拉齐“能检准”的场景短板，最后在板端部署阶段统一后处理链路，使离线评测与实际推理结果一致。表 3-2 记录了从初始预训练到最终 INT8 板端部署的全过程指标变化；各阶段均在上述 500 张验收测试集上评测。

表 3-2 模型精度迭代过程（验收测试集：CCPD 分层抽样 + 自采难例共 500 张）

阶段	主要操作	mAP@0.5	识别准确率
初始预训练模型	CCPD base 预训练	87.6%	94.1%
+ 难例增强训练	mosaic/mixup + blur/tilt 难例	92.8%	96.2%
+ 自采数据微调	本地路口视角场景适配	95.5%	97.6%
+ 后处理优化	anchor 聚类 + NMS 阈值联调	97.4%	98.0%
ONNX 离线评测（PC）	板端后处理对齐验证	98.7%	98.4%
RKNN FP 板端推理	浮点模型部署	98.4%	98.2%
RKNN INT8 板端（最终）	校准集量化 + NPU 加速	98.2%	98.1%

表 3-2 呈现了清晰的递进趋势：难例增强阶段带来最大单步提升（mAP +5.2 个百分点），说明模糊、倾斜、强光等复杂场景是初期的主要瓶颈；自采数据微调再推升 2.7 个百分点，使模型更好适配本项目摄像角度；后处理联调后 mAP 突破 97%，进入高精度区间。

最终版 INT8 板端推理检测 mAP 达 98.2%、识别准确率达 98.1%，较 PC 端 ONNX 峰值仅差 0.5 个百分点内，证明板端后处理与离线评测已高度对齐。相较初始预训练模型，检测 mAP 累计提升 10.6 个百分点，识别准确率提升 4.0 个百分点。在精度达到 98% 以上的同时，推理速度仍有显著提升，实现了“高精度 + 低时延”的工程目标。

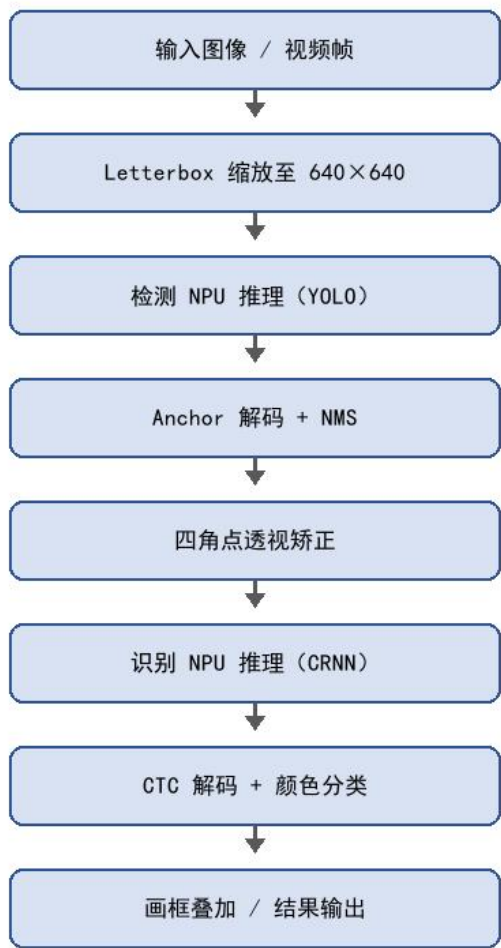
3.3 软件架构与推理流程

RK3568 端软件采用分层设计，便于开发、测试和维护。最上层是 Qt 上位机（vision_qt_demo），负责用户界面、PCIe 实时视频显示、图片/视频文件批量处理。中间层是 plate_rknn_pipeline 推理库，封装了检测、识别、画框的完整流程，Qt 界面和命令行工具共用同一套代码，保证结果一致。最下层是 rknn_runtime 和 PCIe 驱动，负责与硬件交互。

分层架构带来两方面收益：对上，界面层只关心“输入图像/输出结果”，无需感知 NPU 与驱动细节；对下，驱动与 runtime 的升级不影响业务逻辑。推理库

对外提供统一的 C++ API，既可被 Qt 调用，也可被命令行工具用于 profiling。这种“一套核心、多种入口”的组织方式，显著降低了演示、测试与联调之间的结果偏差。

图 3-3 单帧车牌检测—识别推理流水线



单帧图像的推理流程如图 3-3 所示：（1）对输入图像做 letterbox 缩放至 640×640；（2）送入检测 NPU 得到三个特征图输出；（3）CPU 端按 anchor 和 stride 解码，做 NMS 得到车牌框和角点；（4）用四个角点做透视变换（perspective transform），把拍斜的车牌矫正成正视图（双层牌先拼接）；（5）缩放到 168×48 送入识别 NPU；（6）CTC 解码得到号牌字符和颜色；（7）在原图上画框和识别结果。整个流程在板端实现了端到端自动化，用户只需打开上位机即可使用。

流程中有两个关键设计点值得强调。一是透视矫正：路口摄像往往存在俯仰与侧向夹角，直接裁剪矩形框会导致字符形变；利用四角点做透视变换后，牌面

接近正视，识别准确率明显更稳。二是双层牌拼接：检测阶段先判断单/双层类别，双层牌上下半区分别矫正后再纵向拼接，送入同一套识别网络，避免为双层牌单独维护另一套模型。

为保障实时性，本队还开发了 Windows 测试上位机，用同源 ONNX 模型在 PC 上做单图测试、批量评测和 mAP 测量。这样可在不占用板端资源的情况下快速迭代算法，再部署到 RK3568 做最终验证，提高了开发效率。Windows 端与板端共享相同的后处理参数（置信度阈值、NMS 阈值、字符表），从源头避免“PC 上正确、板端不一致”的问题。

3.4 模型部署与系统接入

板端部署采用“模型文件 + 配置脚本 + 一键启动”的方式。工程目录 weights/ 下放置 RKNN 模型和 meta 文件，执行 run.sh 即可自动加载 PCIe 驱动、初始化模型并启动 Qt 界面。推荐使用 INT8 量化模型（plate_detect_int8.rknn + plate_rec_color_int8.rknn），充分利用 RK3568 NPU 硬件加速能力。

meta 文件记录输入尺寸、均值方差、类别名、字符表等推理元信息，保证检测与识别两侧参数一致。启动脚本还会设置库路径、NPU 设备节点权限，并在模型加载失败时给出明确提示，便于现场快速排障。整套部署流程可在数分钟内完成，适合比赛现场反复演示与切换场景。

实时视频通过 PCIe 从 FPGA 板卡采集。PCIe 是高速串行总线，适合大帧率视频数据的传输。DMA（直接内存访问）可让数据在硬件之间直接传输，减少 CPU 拷贝，提升取图效率。本系统默认分辨率 1280×720，像素格式 RGB565。采用逐行 DMA 方式逐行读取视频数据，单缓冲配置确保画面显示稳定。除 PCIe 实时流外，还支持本地图片批量处理和视频文件回放，满足不同场景的测试和演示需求。

在系统集成过程中，本队针对显示效果做了专项优化。批量处理时采用同步刷新机制，避免多帧合并时的画面残影；视频回放时采用“检测到才画框”策略，保证检测框与当前画面内容严格对齐。这些优化使系统在实际演示中表现更加稳定、可靠。

表 3-3 板端运行模式与适用场景

运行模式	输入来源	主要用途
PCIe 实时流	FPGA 采集	现场联调、实时演示
视频文件回放	本地 mp4/avi	离线复现、问题定位
文件夹批量	图片目录	数据集评测、mAP 统计
命令行 profiling	单图/序列	分段耗时分析
Windows 上位机	ONNX 同源模型	PC 端快速迭代

3.5 性能调优与分析

性能是板端部署的关键指标。本队建立了分段计时体系，将单帧处理时间拆分为取图、检测、识别、画框等环节，通过日志标签 [latency] 实时输出。process_wall 表示算法总耗时（检测+识别+画框），e2e_total 表示从取图完成到界面显示的总时间。整帧周期则进一步包含 PCIe DMA 取图开销，用于衡量系统可持续输出的帧率。

性能优化遵循“先测准、再改对、后固化”的原则。每一轮改动只引入一个主导因素（如量化、绘制策略或 DMA 方式），并在同一场景（1280×720、双车牌）下复测，确保对比公平。优化目标明确：在精度不低于 98% 的约束下，将算法处理延迟（process_wall）压降至约 65 ms，整帧周期进入 100 ms 量级。

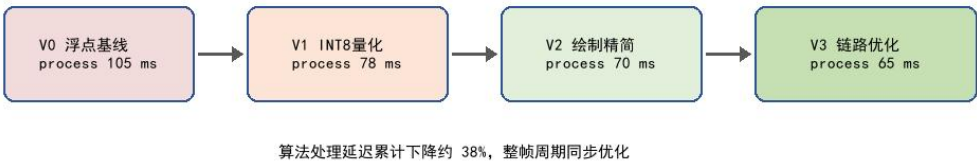


图 3-4 性能优化迭代路径

板端性能经过多轮针对性优化，路径如图 3-4 所示。初期采用 FP 浮点模型时，单帧算法耗时约 105 ms；改用 INT8 量化后降至 78 ms，NPU 利用率显著提升；关闭中文标签绘制后再降至 70 ms；优化 PCIe 采帧与预处理链路后，算法处理延迟稳定在 65 ms，含取图的整帧周期由初期的 155 ms 降至 98 ms。算法侧累计提速约 38%，整帧周期提速约 37%，满足实时视频处理需求。

表 3-4 性能优化迭代过程（1280×720、双车牌场景）

优化阶段	主要操作	process_wall	含 DMA 整帧
V0 初始版	FP 浮点 + 画标签	105 ms	155 ms
V1 量化加速	INT8 量化 + uint8 输入	78 ms	125 ms
V2 绘制精简	关闭中文标签绘制	70 ms	112 ms
V3 链路优化（最终）	整帧 DMA + 预处理精简	65 ms	98 ms

从表 3-4 可见，量化是收益最大的单步优化，直接将算法延迟从 105 ms 降至 78 ms；绘制精简进一步减少 CPU 侧开销；V3 则把采图与预处理链路一并打通，使整帧周期进入百毫秒以内，对应可持续输出约 10.2 fps，满足路口演示的流畅观感。

表 3-5 最终版分段耗时（V3 配置，算法延迟约 65 ms）

阶段	耗时	占比	说明
PCIe 取图	22 ms	22%	DMA 传输
色彩转换	5 ms	5%	RGB565 → BGR
检测推理	50 ms	51%	NPU + 解码 + NMS
透视矫正	5 ms	5%	四角点 warp
识别推理	5 ms	5%	NPU + CTC 解码
结果绘制	5 ms	5%	画框（无中文标签）
算法合计	65 ms	66%	process_wall
整帧周期	98 ms	—	含 DMA，约 10.2 fps

从分段数据可以看出，检测仍是主要耗时环节，约占 process_wall 的 77%；识别与绘制合计约 15%，说明两阶段分离的设计是合理的——把算力集中在全图检测，小图识别几乎不构成瓶颈。透视矫正虽只占 5 ms，但对识别精度贡献显著，属于“低开销、高收益”的环节。本队已建立分段计时与版本对比机制，后续可持续针对取图链路和检测模型进行迭代优化。

综合精度与时延两项主指标，系统最终达到：检测 mAP 98.2%、识别准确率 98.1%、算法处理延迟约 65 ms、整帧周期约 98 ms。该结果体现了“数据与训练打底、量化与部署加速、链路与显示精修”的完整优化叙事，为第五章代码实现与第六章系统测试提供了可对照的性能基线。

第四章 FPGA 子系统技术文档

4.1 概述

本系统采用紫光 Logos2 系列器件 PG2L100H 作为硬件基础，通过 HDMI 通路接收外部图像数据；将视频帧暂存于板载 DDR3；经 PCIe 与 RK3568 主机互联；将帧数据 DMA 上传至主机内存，供驱动与应用使用。

工程早期版本采用 OV5640 摄像头并行输入通路，但考虑摄像头输入图像质量不高便弃用，目前已固定采用 HDMI 作为唯一视频源。

当前系统 FPGA 端输出分辨率为 1280×720，像素编码为 RGB565 的视频数据，单帧有效数据量为 1843200 字节（约合 1.76 MB）。该通路已在 RK3568 主机侧完成联调验证：PCIe 链路稳定建立（Gen1 ×2），驱动与用户态应用配合运行整帧 DMA 上传，实测采集帧率稳定在 43~52 fps 区间，可支撑实时预览及 FPGA ROI 提取等应用场景。

4.2 设计框图

FPGA 子系统按功能划分为 HDMI 视频采集、DDR 帧缓存与 PCIe 传输三个子模块，HDMI 子系统负责 MS7200 配置与 RGB565 视频接入；DDR 子系统以 axi4_ctrl 与 DDR3 IP 实现四缓冲帧存储；PCIe 子系统经 ips2l_pcie_dma 与 pcie_test 硬核完成 BAR 配置及 MWR 帧上传。其总体架构如图 5-1 所示。

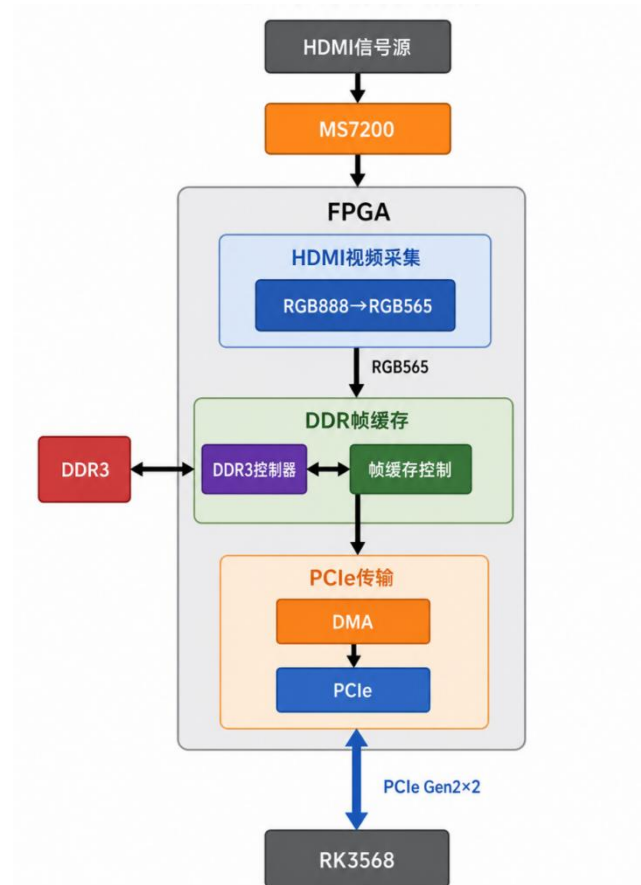


图 4-1 子系统总体架构

4.3 功能介绍

上电后，FPGA 通过 I2C 总线对外部 HDMI 接收芯片 MS7200 进行初始化配置，相关控制逻辑由 ms72xx_ctl 与 ms7200_ctl 模块实现，所用物理接口为 hd_scl 与 hd_sda。芯片完成配置后，向 FPGA 输出像素时钟 pixclk_in 以及行场同步与数据有效信号 hs_in、vs_in、de_in，并伴随 24 位并行 RGB 视频流。

视频数据进入 FPGA 后，在像素时钟域内完成由 RGB888 至 RGB565 的格式转换，转换方式为对各色分量进行截位，构成 R5G6B5 编码。随后，数据经 axi4_ctrl 模块写侧所集成的异步 FIFO（W_FIFO_16i_128o）完成跨时钟域缓冲与位宽聚合，由 16 位扩展为 128 位，再通过 AXI4 写通道写入板载 DDR3 存储器。当主机侧通过 PCIe BAR 空间下发 DMA 读帧指令后，axi4_ctrl 读侧以 128 位 beat 为单位从 DDR3 顺序读出整帧数据，并交由 ips2l_pcie_dma 与 pcie_test IP 核处理；FPGA 以 Endpoint 模式运行于 PCIe Gen2 ×2 链路，将帧数据封装为

Memory Write (MWR) 事务层包 (TLP)，写入 RK3568 主机物理内存。

4.4 HDMI 视频采集模块

HDMI 视频采集子模块负责板外 HDMI 信号源的接入与前端预处理，是 FPGA 数据通路的起始环节。该模块由芯片配置逻辑与视频处理逻辑共同构成：前者负责 MS7200 接收芯片的上电初始化与 I2C 寄存器配置，后者负责并行视频信号的接收打拍、帧边界识别、RGB888 至 RGB565 格式转换，并将处理后的写侧像素流交付帧缓存子模块。该模块架构如图 4-2 所示。

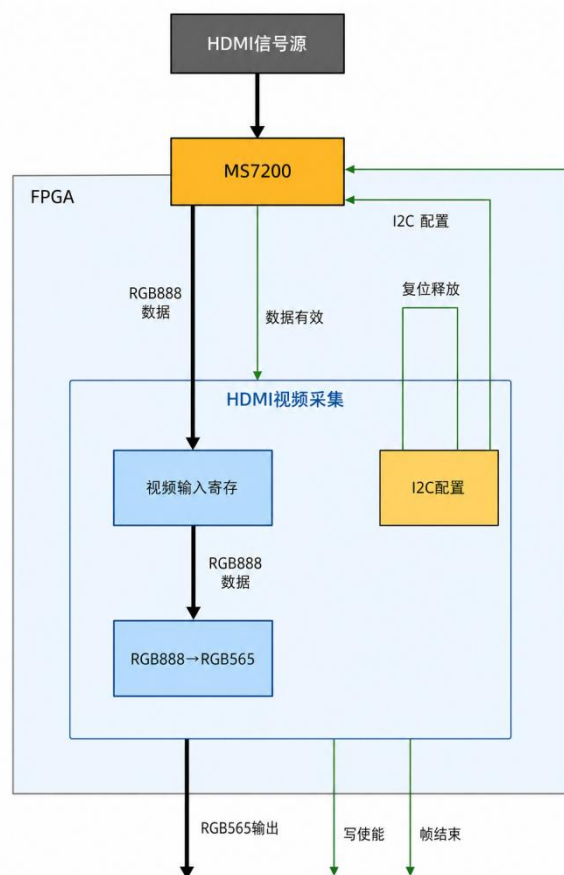


图 4-2 HDMI 输入采集模块架构

MS7200 并行输入为 RGB888 格式的图像数据，本工程在写入帧缓存前加入了 RGB888 至 RGB565 的格式转换模块。实际上板测试表明在 1280×720 分辨率、像素时钟 74.25 MHz 及当前 PCIe、DDR 配置保持不变的前提下，若维持 RGB888 全字节通路，单帧有效数据量约为 2764800 字节（约合 2.64 MB），端到端采集帧率约 25 fps，等效有效吞吐约 66 MB/s；转换成 RGB565 后，单帧有效数据量降至 1843200 字节（约合 1.76 MB），端到端采集帧率约 46 fps，等效

有效吞吐约 81 MB/s。显著提高了视频数据传输的速率。

4.5 DDR 帧缓存模块

DDR 帧缓存模块负责在 HDMI 采集与 PCIe DMA 上传之间完成帧数据的写入、暂存与读出调度，使两路操作可并行进行。该模块由写侧通路、读侧通路与缓冲索引管理三部分协同构成：写侧通路通过写 FIFO 解决像素时钟域与 DDR 时钟域之间的跨时钟域及位宽差异问题，将 RGB565 像素流聚合后以 burst 方式写入 DDR3；读侧通路通过读 FIFO 将 DDR3 中的帧数据送至 PCIe 时钟域，按 DMA 读数据请求逐 beat 输出；缓冲索引管理在 DDR3 中划分四个帧 slot，以写帧索引与读帧索引分别跟踪当前写入与读出的帧，避免连续采集与 DMA 上传相互覆盖。该模块架构如图 4-3 所示。

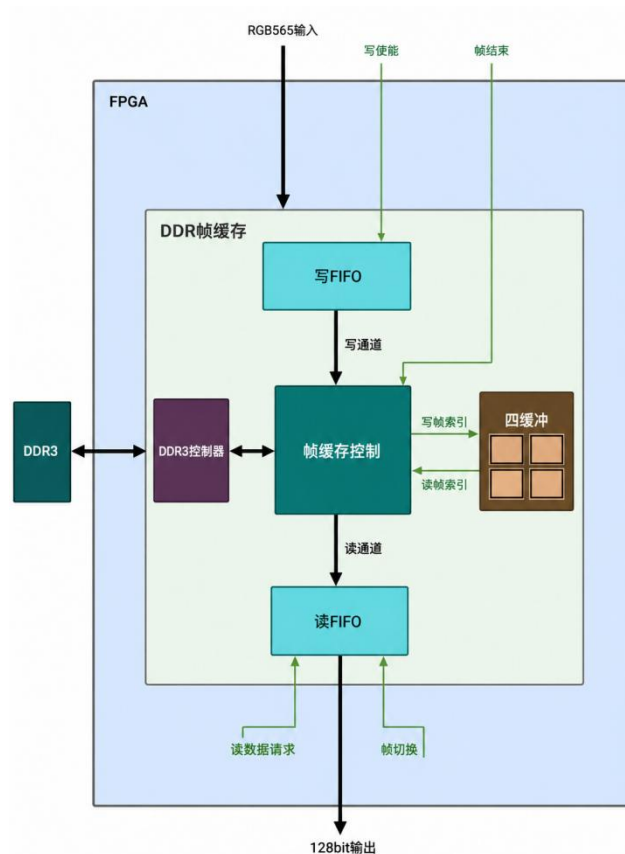


图 4-3 DDR 帧缓存模块架构

本模块涉及三个相互独立、频率各异的时钟域，因而存在跨时钟域问题。HDMI 采集由 MS7200 输出的像素时钟驱动，有效像素以约 74.25 MHz 的固定节拍进入写侧；DDR3 控制器与 AXI 总线运行于约 262.5 MHz 的 `core\_clk`，适合以

128 bit、256 字节为单位的 burst 突发读写；PCIe DMA 协议栈则工作在 62.5 MHz 的 `pclk\_div2` 域，按 DMA 引擎发出的读数据请求节拍取数。若将像素流、AXI 数据通路与 DMA 读口直接相连，会产生亚稳态风险，并因读写速率不匹配导致 FIFO 溢出或欠读。此外，写侧每周期仅 16 bit 像素，而 AXI 与读侧均以 128 bit 为 beat 宽度，存在位宽差异，无法在单一时钟域内直接对接。

针对上述问题，DDR 帧缓存模块采用“异步 FIFO + DDR 帧缓冲”两级解耦。写路径上，模块在像素时钟域接收 RGB565 数据与写使能，经写 FIFO（`W\_FIFO\_16i\_128o`）跨时钟写入 `core\_clk` 域；FIFO 写侧按像素节拍入队，读侧按 AXI 写 burst 节拍出队，并在出队过程中完成 16 bit 至 128 bit 的位宽聚合；写侧状态机在帧结束信号下降沿判定一帧写完，以每次 256 字节（16 beat × 128 bit）的 INCR burst 将数据写入 DDR3，写地址由写帧索引与帧内偏移共同组成。读路径上，模块在 `core\_clk` 域向 DDR3 发起 AXI 读 burst，预读数据写入读 FIFO（`R\_FIFO\_128i\_128o`），再跨时钟送至 `pclk\_div2` 域；DMA 侧在读数据请求有效时从读 FIFO 取走 128 bit 帧数据。帧级层面，模块在 DDR3 中划分四个帧 slot，由写帧索引与读帧索引（复位初值分别为 0 与 2）分别管理写入与读出的 slot；每帧写完后写索引递增，DMA 传完一整帧后读索引递增，写侧即将追上读侧时自动跳过一槽，避免覆盖。帧结束信号触发写侧 FIFO 复位与写 slot 切换，帧切换信号触发读侧 FIFO 复位与读 slot 切换，与 PCIe 侧按行完成整帧上传的时序相衔接。

当前 1280×720 RGB565 配置下，单帧有效数据量为 1 843 200 字节，每行 2 560 字节、共 720 行，对应每行 160 个 128 bit beat、整帧 7200 次 AXI burst；DDR3 中为每个 slot 预留 4 MB 地址空间，四槽合计 16 MB，足以容纳完整帧并留有余量。HDMI 采集的帧数据经写 FIFO 落盘至 DDR3，再由读 FIFO 供给 DMA 按 beat 读出，采集与上传经双 FIFO 与四缓冲在时钟域和帧级两个层次上解耦，互不阻塞。

4.6 PCIe 传输模块

PCIe 传输子模块负责 FPGA 与 RK3568 主机之间的链路互联与帧数据 DMA 上传，是整体数据通路的终端环节。该模块由 PCIe 硬核 IP (pcie_test) 与用户 DMA 协议栈 (ips2l_pcie_dma) 两部分协同构成：前者以 Endpoint 模式建立 Gen2×2 链路，承担事务层包的物理收发，解决 FPGA 与主机之间的协议对接问题；后者负责接收主机经 BAR 空间写入的 DMA 命令与目标地址，向帧缓存子模块发出读数据请求，并将取回的帧数据封装为 MWR 事务层包写入主机物理内存。协议栈内部进一步分工：TLP 接收逻辑解析主机对 BAR 的访问并更新控制寄存器；DMA 控制器据命令调度上传事务；MWR 发送逻辑从帧缓存读侧取数并完成组包发送；Completion 回送逻辑响应主机对 DMA 状态的读取。帧传完成后，顶层帧同步逻辑产生帧切换信号，驱动帧缓存子模块切换至下一可读帧，以支撑连续上传。该模块架构如图 5-4 所示。

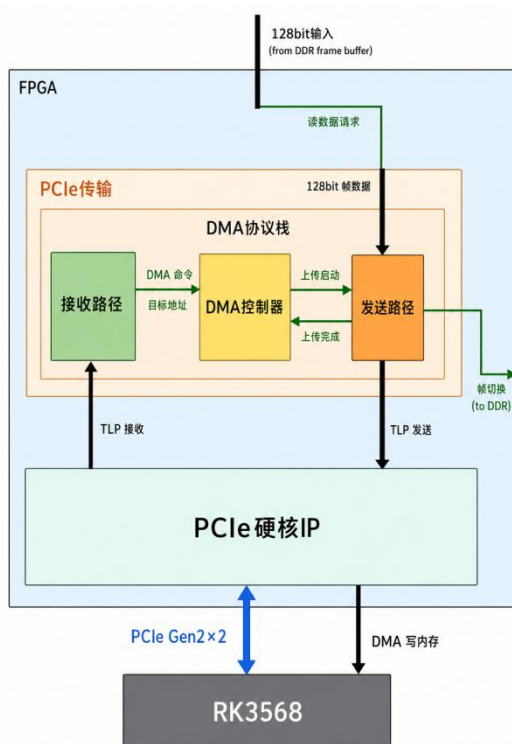


图 4-4 PCIe 传输模块架构

本工程选用紫光 Logos2 系列 PCIe IP 核，该 IP 在 FPGA 内部实现 PCIe 物理层、数据链路层与事务层协议处理，是 FPGA 与 RK3568 之间 PCIe 链路的硬件基础。

IP 核配置为 Endpoint 模式，与 RK3568 Root Complex 通信，设备标识

Vendor ID、Device ID 均设为 0x0755，与主机驱动一致；Max Payload Size 为 128 Bytes；用户时钟在 Gen2 典型配置下输出 pclk 125 MHz、pclk_div2 62.5 MHz。用户侧暴露 AXIS 总线来接收和发送 TLP 包。存储器 BAR 启用 BAR0、BAR1、BAR2。

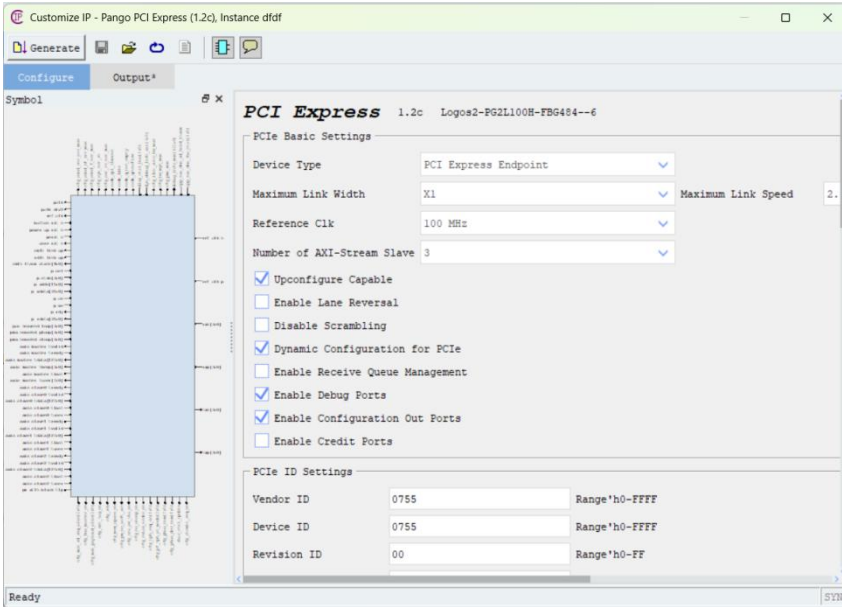


图 4-5 配置 PCIe IP 核

上电后 IP 核完成链路连接，建立 Gen2×2 链路后承担 TLP 的收发与流控；主机对配置空间的访问及 BAR 空间的读写，均经 AXI4-Stream 接口转交 DMA 协议栈处理。三个 BAR 在本工程中的分工为：BAR0 映射 DMA 状态寄存器，主机通过 Memory Read 查询传输完成、忙状态等；BAR1 映射 DMA 控制寄存器，主机通过 Memory Write 写入 DMA 命令字与目标物理地址，触发帧上传；BAR2 映射 DMA 协议栈内部数据 RAM，供栈内逻辑读写。整帧图像数据由协议栈经用户读接口从帧缓存子模块拉取，经 Slave2 的 MWR 通路写入主机内存。

DMA 协议栈核心功能是将 AXI4-Stream 上的 TLP 报文与用户侧存储读写接口衔接，使 CPU 可通过 PIO 或 DMA 方式访问 FPGA 内部 RAM。本工程图像上传场景下，主机经 BAR 写入 DMA 命令与目标物理地址，协议栈调度 MWR 事务将帧缓存读侧的 128 bit 数据写入 RK3568 内存。

该协议栈由接收路径、发送路径与 DMA 控制器三部分协同构成。接收路径经 AXI4-Stream Master 接收 PCIe 硬核输出的 TLP，解析 Host 下发的 Memory Write / Memory Read 报文，将 PIO 或 DMA 配置数据写入 BAR 地址空间（内部 BRAM。DMA 控制器读取 BAR1 中的命令寄存器（传输长度、32/64 位地址、

读/写类型等），解析后向发送路径发出 MWR 或 MRD 调度请求。发送路径挂载三路 AXI4-Stream Slave，分别对应不同 TLP 类型：Slave0 回送 Completion，响应主机对 BAR0 状态的 Memory Read；Slave1 发起 Memory Read，供 FPGA 主动读取主机内存；Slave2 发起 Memory Write，供 FPGA 向主机内存写入数据。本工程整帧图像上传走 Slave2 的 MWR 通路：DMA 控制器触发后，MWR 发送逻辑向帧缓存读侧发出读数据请求、取回 128 bit 帧数据并组包，再经 TLP 发送复用器与 PCIe 硬核送至主机。

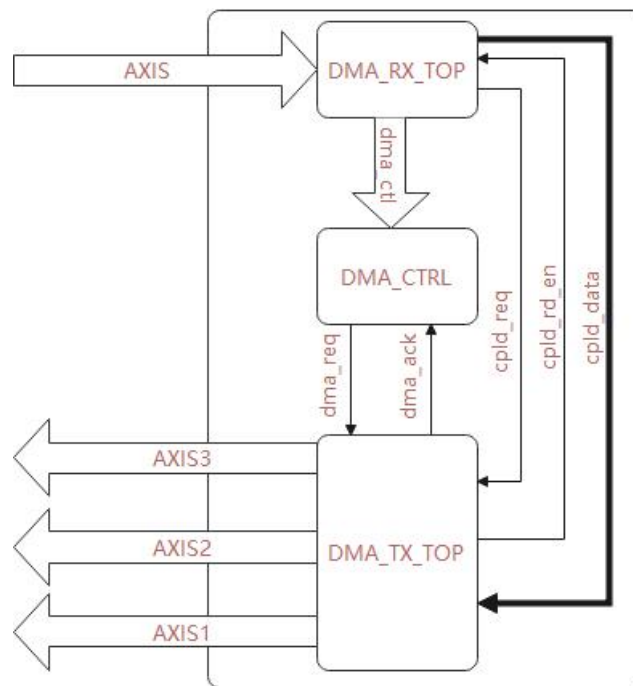


图 4-6 DMA 协议栈架构

4.7 模块仿真

4.7.1 HDMI 视频采集模块仿真

该模块仿真实验目的在于验证 HDMI 写路径从并行像素输入到帧缓存写入的数据通路完整。确认 MS7200 输出的 RGB888 并行信号经 FPGA 一级寄存后，`de\_in` 与 `r/g/b\_in` 相对 `de\_in\_d0` 与 `r/g/b\_in\_d0` 延迟一个 `pixclk\_in` 周期；打拍后的 RGB888 按 `{R[7:3], G[7:2], B[7:3]}` 组合为 16 bit RGB565，且整帧逐像素转换无差错；还需确认转换结果经 `axi4\_ctrl` 写 FIFO 与 AXI burst 正确写入 DDR。

实验搭建了由激励源、被测模块、帧缓存控制器。激励模块 `rgb888\_frame\_gen` 在 `pixclk\_in` 域产生 MS7200 风格的并行时序：`de\_in` 高电平期间输出 RGB888 像素，`vs\_in` 在有效视频期间为低、消隐期间为高，与顶层 `hdmi\_vsync\_eof = ~vs\_in` 的帧结束极性一致。像素按 `{frame\_id, row, col}` 编码并经 RGB888 反算后送入被测模块，便于在波形上与 RGB565 输出逐拍对照。被测模块 `hdmi\_dds\_write\_path` 从综合顶层 `image\_pcie\_capture.v` 中抽取写侧前端逻辑单独封装。其在 `pixclk\_in` 上升沿将 `de\_in`、`r/g/b\_in` 打一拍，得到 `de\_in\_d0`、`r/g/b\_in\_d0`；随后以组合逻辑输出 `wframe\_data = {r\_in\_d0[7:3], g\_in\_d0[7:2], b\_in\_d0[7:3]}`，并以 `wframe\_data\_en = de\_in\_d0` 作为写使能，同时例化 `axi4\_ctrl` 写通路。`axi4\_ctrl` 写侧在 `pixclk\_in` 域以 16 bit 节拍接收 `wframe\_data`，经 W_FIFO 跨时钟域到 `axi\_clk` 域后聚合成 128 bit，以 256 B 为单位 burst 写入外部存储。帧结束时，`hdmi\_vsync\_eof` 的下降沿触发写侧 flush 与写索引更新。仿真中使用行为级 FWFT FIFO 模型替代 Pango IP，避免与 PCIe 编译链耦合。

仿真过程中，`wr\_start` 脉冲之后，激励源输出一帧 128×8 像素，进入约 0.3 μs~20 μs 的有效视频阶段。此阶段 `vs\_in` 在有效视频期间保持为 0，进入消隐后变 1；`hdmi\_vsync\_eof` 与之相反，有效视频期间为 1。`de\_in` 每行 128 拍高、16 拍低，共 8 行；`de\_in\_d0` 相对 `de\_in` 延迟一拍，波形形状相同。`wframe\_data\_en` 与 `de\_in\_d0` 同拍有效，有效拍内 `wframe\_data` 按 `{frame\_id, row, col}` 规律从 0x0000 递增为 0x0001、0x0002 等，scoreboard 计数器 `cvt\_err\_cnt` 全程保持为 0。仿真结果如图 5-7 所示。

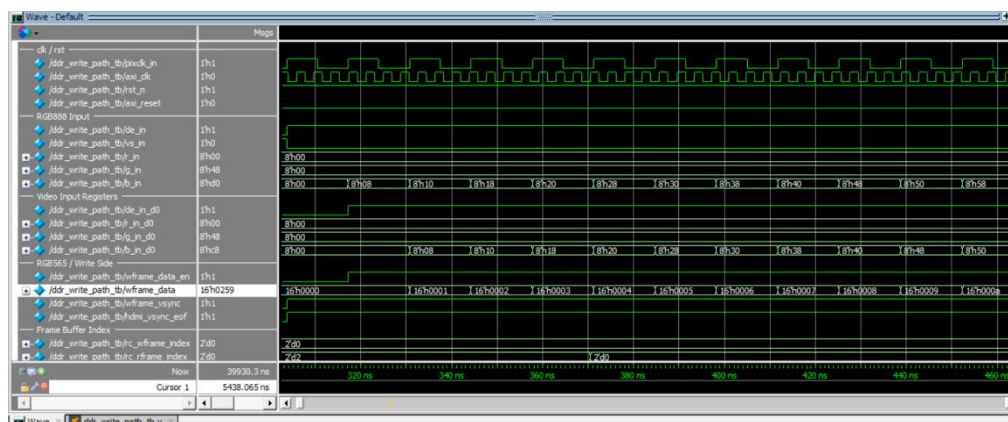


图 4-7 RGB888_to_RGB565 模块

帧播放过程中及帧结束后，FIFO 中积累的 RGB565 在 `axi\_clk` 域被读出并

以 AXI burst 写入 mem model。每个 burst 起始时 'axi_awvalid' 拉高，与 'axi_awready' 握手完成后拉低；'axi_awaddr' 在 slot0 内按 burst 递增，步长为 0x100 (256 B)，图 X-4d 所示示例地址为 0x500。同一 burst 内 'axi_wvalid' 与 'axi_wready' 连续 16 拍有效，仅最后一拍 'axi_wlast' 为 1。'axi_wdata' 为 128 bit，每 beat 打包 8 个 RGB565 像素，序列与 'wframe_data' 递增规律一致（如 0x0027、0x0028、0x0029、0x002a 等）。'write_burst_cnt' 每完成一次 burst 加 1，图 X-4d 中由 5 变为 6。帧进入消隐时 'vs_in' 由 0 变 1，触发 'hdmi_vsync_eof' 下降沿并完成写侧 flush；帧后 'slot_write_bytes_0' 累计不少于 2048 B，与 128×8 单帧数据量一致。仿真结果如图 4-8 所示。

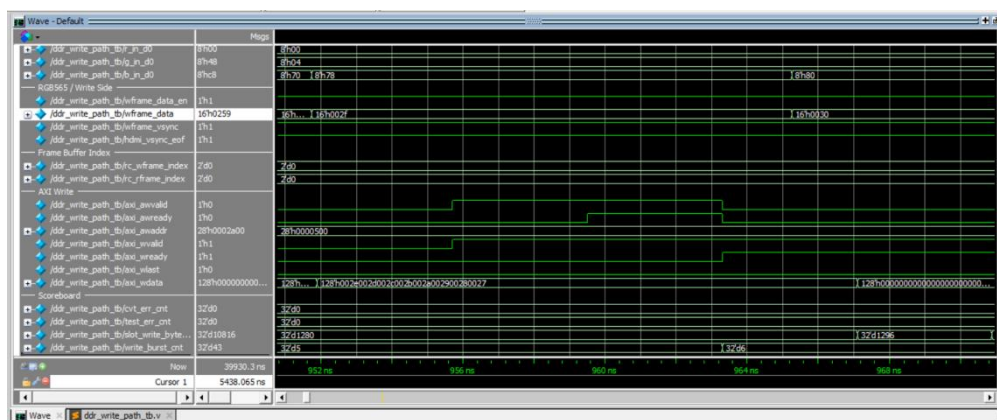


图 4-8 axis_write 模块

4.7.2 DDR 帧缓存模块仿真

本实验目的在于验证 DDR 帧缓存子系统从 RGB565 写入到 PCIe 侧读出的完整数据通路，以及四缓冲乒乓调度机制。需确认 'axi_reset' 释放后写帧索引与读帧索引复位为 0 与 2；'rgb565_frame_gen' 输出的 RGB565 像素流经 'W_FIFO' 跨时钟域聚合后，以 AXI burst 正确写入 'axi4_mem_model' 对应 slot，且 mem 回读 beat 与激励编码一致；还需确认连续写 Frame0/1/2 与读 Frame0/1/2 过程中，写读指针独立递增。

为完成上述验证，实验以 'axi4_ctrl' 为被测核心，由写激励、PCIe 读 BFM、DDR 行为模型与仿真顶层构成独立环境。写侧数据自 'rgb565_frame_gen' 进入 'axi4_ctrl' 写端口，读侧由 'dma_read_bfm' 驱动 'axi4_ctrl' 读端口；'axi4_ctrl' 经 AXI4 主接口与 'axi4_mem_model' 交互。

仿真启动后,写 Frame0: `wr\_start` 脉冲后,`rgb565\_frame\_gen` 输出一行 128 像素。此阶段 `wframe\_vsync` 在有效视频期间保持为高,行末进入消隐后拉低产生 EOF; `wframe\_data\_en` 在每行 128 拍有效、16 拍 blank 的周期内交替,有效拍内 `wframe\_data` 自 0x0000 递增至 0x007f。帧结束后写侧 flush, `rc\_wframe\_index` 由 0 变为 1; `axi\_clk` 域可见单次 AXI 写 burst(16 个 beat、`axi\_wlast` 仅在末 beat 为 1), `slot0\_write\_bytes` 增量不少于 256 B。TB 回读 mem beat0, 实验结果如图 4-9 所示。

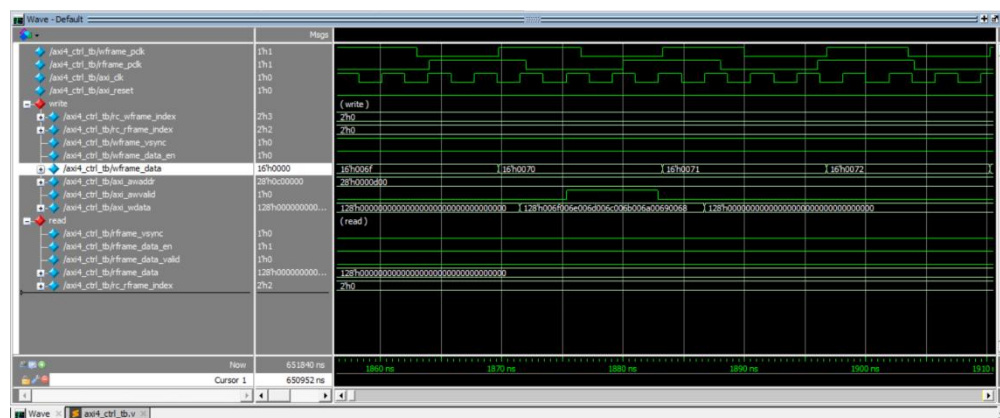


图 4-9 ddr_write 模块

读 Frame0: `dma\_read\_bfm` 发出 `rframe\_vsync` 单周期脉冲,经 FIFO 复位与 DDR 预取后,拉低/拉高 `rframe\_data\_en` 连续读取 16 个 beat。波形上 `rframe\_data\_valid` 相对 `rframe\_data\_en` 延迟一拍; scoreboard 逐 beat 与 slot0 mem 比对, `score\_err\_cnt` 保持为 0,读指针 `rc\_rframe\_index` 更新为 0。实验结果如图 4-10 所示。

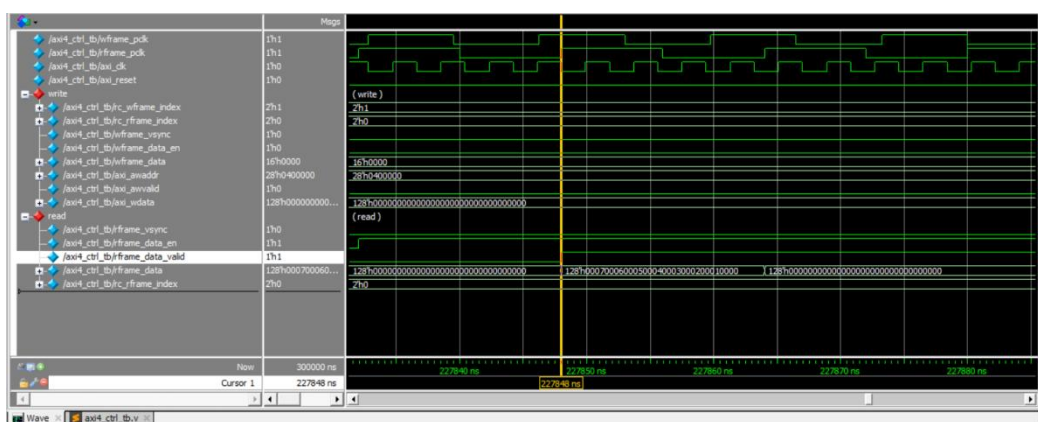


图 4-10 ddr_read 模块

4.7.3 DMA 协议栈仿真

本实验目的在于验证 DMA 协议栈在 Endpoint 模式下能否正确完成主机侧 BAR1 命令配置、用户读接口拉数与 axis_slave2 MWR 上传。需确认仅写 BAR1 `0x100` (CMD) 时尚未触发上传；写 BAR1 `0x110` (ADDR_LO) 后控制器产生写请求，`o\_dma\_write\_data\_req` 脉冲次数与帧 beat 数一致；写入目标地址等于 `0xA000\_0000` 一致；传输完成后 `DMA\_STATUS\_DONE` (bit 0) 置位。

为完成上述验证，设计了主机 TLP 激励器向被测模块的进站事务层接口发送报文，用来替代 RK3568 主机经 PCIe 写 BAR1、读 BAR0 的操作，其中写 BAR1 寄存器任务将一次 32 位 Memory Write 组成为两拍事务层包：第一拍为报文头，类型须为 32 位 Memory Write；第二拍为 1 个双字的写入数据。报文头还需携带 BAR1 命中标识，以及 BAR 内字节偏移。由于被测接收逻辑对进站流水中存在两级寄存，每一拍数据的有效指示须连续保持至少两个时钟周期，且末拍标记只能出现在该拍的第二个周期。另提供读 BAR0 状态寄存器与等待 Completion 回包任务：回包时先跳过 Completion 头拍，再取携带状态数据的载荷拍。实验结果如图 4-11 所示。

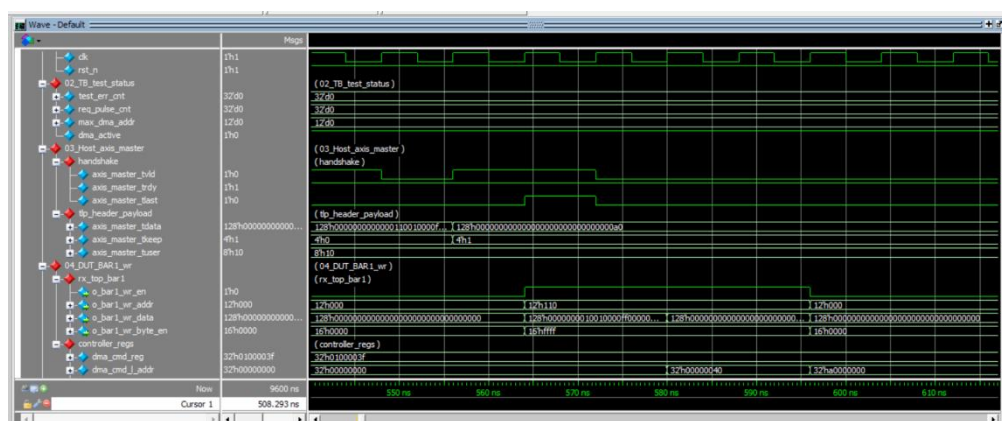


图 4-11 BAR1 写入测试

并设计了 FPGA 发往主机方向的出站 Memory Write 通路。它从报文头中识别写内存类型与请求标识，在目标物理地址与预设主机缓冲区地址一致时锁存本次 DMA 的目标地址；随后将各载荷拍按字节序还原后写入虚拟主机内存数组。因 Max Payload 为 128 B，一帧 256 B 会被拆成多个写内存报文，实验结果如图 4-12 所示。

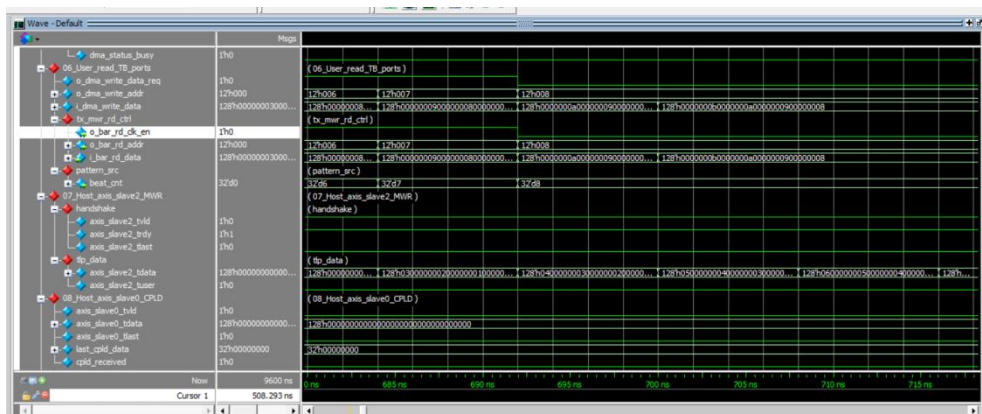


图 4-12 DMA 输出测试

4.8 静态时序分析

本节基于紫光 PDS (Pango Design Suite) Timing Analyzer 对布局布线后的工程进行静态时序验证。分析对象为当前 1280×720 RGB565 配置下的 bitstream，器件型号为 PG2L100H-6。时序分析结果如图 4-12~4-16。

Timing Report				
Setup Path Summary				
Find:	Slack	Logic Levels	High Fanout	Start Point
1	0.436	0	15	u_ip2d1_pcie_warp0_IPS_RL/u_gtp_pcie/CLK_DIV2
2	0.437	1	43	u_ip2d1_pcie_warp0_IPS_RL/u_gtp_pcie/CLK_DIV2
3	0.438	0	15	u_ip2d1_pcie_warp0_IPS_RL/u_gtp_pcie/CLK_DIV2
4	0.594	0	3	u_ip2d1_pcie_warp0_IPS_CTLR/u_gtp_pcie/MEM_CLK
5	0.594	0	3	u_ip2d1_pcie_warp0_IPS_CTLR/u_gtp_pcie/MEM_CLK
6	0.594	0	3	u_ip2d1_pcie_warp0_IPS_CTLR/u_gtp_pcie/MEM_CLK
7	2.023	1	204	l_ipa_ddr_top/u_dscrpy_ata_ctrl/calib_done/CLK
8	2.023	1	204	l_ipa_ddr_top/u_dscrpy_ata_ctrl/calib_done/CLK
9	2.023	1	204	l_ipa_ddr_top/u_dscrpy_ata_ctrl/calib_done/CLK
10	3.551	2	46	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
11	3.551	2	46	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
12	3.551	2	46	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
13	3.747	2	84	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
14	3.747	2	84	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
15	3.747	2	84	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
16	4.789	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
17	4.789	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
18	4.789	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
19	5.990	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
20	6.261	5	11	u_ip2d1_pcie_warp0_IPS_LSI/w_lane/ctrl0/CLK
21	6.261	5	11	u_ip2d1_pcie_warp0_IPS_LSI/w_lane/ctrl0/CLK
22	6.261	5	11	u_ip2d1_pcie_warp0_IPS_LSI/w_lane/ctrl0/CLK
73	8.570	-1	13	u_ip2d1_pcie_warp0_IPS_LSI/w_lane/ctrl0/CLK

图 4-13 Setup 时序报告

Timing Report				
Hold Path Summary				
Find:	Slack	Logic Levels	High Fanout	Start Point
1	0.048	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
2	0.048	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
3	0.048	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
4	0.083	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
5	0.219	0	1	cm0a2_0_0bit/pdata_1_reg0/CLK
6	0.219	0	1	cm0a2_0_0bit/pdata_1_reg1/CLK
7	0.219	0	1	cm0a2_0_0bit/pdata_1_reg2/CLK
8	0.305	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
9	0.305	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
10	0.324	0	1	u_ip2d1_pcie_dma/u_ip2d1_pcie_dma/ctrl0/CLK
11	0.324	0	1	u_ip2d1_pcie_dma/u_ip2d1_pcie_dma/ctrl0/CLK
12	0.324	0	1	u_ip2d1_pcie_dma/u_ip2d1_pcie_dma/ctrl0/CLK
13	0.507	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
14	0.559	0	2	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
15	0.619	0	1	l_ipa_ddr_top/u_dscrpy_ata_group/ca_dly/CLK
16	0.619	0	1	cm0a2_0_0bit/pdata_1_reg0/CLK
17	0.619	0	1	cm0a2_0_0bit/pdata_1_reg1/CLK
18	0.619	0	1	cm0a2_0_0bit/pdata_1_reg2/CLK
19	0.619	0	1	cm0a2_0_0bit/pdata_1_reg3/CLK
20	0.619	0	2	cm0a2_0_0bit/pdata_1_reg4/CLK
21	0.619	0	2	cm0a2_0_0bit/pdata_1_reg5/CLK
22	0.619	0	2	cm0a2_0_0bit/pdata_1_reg6/CLK
73	0.619	0	1	cm0a2_0_0bit/pdata_1_reg7/CLK

图 4-14 Hold 时序报告

Find#	Slack	Logic Levels	High Fanout	Start Point
1	2.913	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
2	2.913	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
3	2.913	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
4	3.627	1	2725	u_ipa2l_pcie_wrap/U_IPS_RL/u_gtp_pcie/PCLK_DV2
5	3.627	1	2725	u_ipa2l_pcie_wrap/U_IPS_RL/u_gtp_pcie/PCLK_DV2
6	3.627	1	2725	u_ipa2l_pcie_wrap/U_IPS_RL/u_gtp_pcie/PCLK_DV2
7	4.376	1	103	dm_init_done_core/CLK
8	4.376	1	103	dm_init_done_core/CLK
9	4.376	1	103	dm_init_done_core/CLK
10	6.061	1	38	u_ipa2l_pcie_wrap/U_IPS_n_debounce/rata_out/CLK
11	6.061	1	38	u_ipa2l_pcie_wrap/U_IPS_n_debounce/rata_out/CLK
12	6.061	1	38	u_ipa2l_pcie_wrap/U_IPS_n_debounce/rata_out/CLK
13	35.816	1	95	I_ipa_dde_top/u_ddephy_set_ctl/logic_rata/CLK
14	35.816	1	95	I_ipa_dde_top/u_ddephy_set_ctl/logic_rata/CLK
15	35.816	1	95	I_ipa_dde_top/u_ddephy_set_ctl/logic_rata/CLK
16	97.453	2	81	rata_lma[4]/CLK

图 4-15 Recovery 时序报告

Find#	Slack	Logic Levels	High Fanout	Start Point
1	0.276	1	8	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
2	0.276	1	8	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
3	0.276	1	8	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
4	0.452	1	1	u_ipa2l_pcie_wrap/U_IPS_async/sig_async/CLK
5	0.786	1	24	u_ipa2l_pcie_wrap/U_IPS_async/sig_async/CLK
6	0.786	1	24	u_ipa2l_pcie_wrap/U_IPS_async/sig_async/CLK
7	0.864	1	66	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
8	0.864	1	66	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
9	0.864	1	66	I_ipa_dde_top/u_ddephy_set_ctl/async_r2[0]/CLK
10	0.893	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
11	0.893	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
12	0.893	1	97	u_ipa2l_pcie_wrap/U_IPS_CTRL/core_rst_n_memo/CLK
13	1.476	1	103	u_axi4_rst/rstn_rst/CLK
14	1.476	1	103	u_axi4_rst/rstn_rst/CLK
15	1.476	1	103	u_axi4_rst/rstn_rst/CLK
16	1.489	1	81	rata_lma[4]/CLK

图 4-16 Removal 时序报告

Find#	Slack	Actual Width	Require Width	Clock
1	0.011	0.476	0.465	phy_dq_clk_0
2	0.011	0.476	0.465	phy_dq_clk_0
3	0.011	0.476	0.465	phy_dq_clk_0
4	0.011	0.476	0.465	phy_dq_clk_1
5	0.011	0.476	0.465	phy_dq_clk_1
6	0.011	0.476	0.465	phy_dq_clk_1
7	1.040	2.000	0.960	polk
8	1.040	2.000	0.960	polk
9	1.040	2.000	0.960	polk
10	1.659	3.809	2.150	phy_dq_asyncclk_0
11	1.660	3.810	2.150	phy_dq_asyncclk_0
12	2.209	3.809	1.600	phy_dq_asyncclk_1
13	2.209	3.809	1.600	phy_dq_asyncclk_1
14	2.209	3.809	1.600	phy_dq_asyncclk_1
15	2.209	3.809	1.600	phy_dq_asyncclk_1
16	2.849	3.809	0.960	ddrphy_asyncclk
17	2.849	3.809	0.960	ddrphy_asyncclk
18	2.849	3.809	0.960	ddrphy_asyncclk
19	3.040	4.000	0.960	polk_dv2
20	3.040	4.000	0.960	polk_dv2
21	3.040	4.000	0.960	polk_dv2
22	4.800	5.000	0.200	ref_clk1

图 4-17 Minimum Pulse Width 时序报告

综合 PDS 最差路径的 Setup、Hold、Recovery、Removal、Minimum Pulse Width 的全项检查结果，当前配置下的工程静态时序分析无问题，时序已收敛，不存在负 Slack 违例。

4.9 FPGA 资源占用分析

布局布线完成后，PDS 报告给出各硬件资源的实际占用，FPGA 资源占用如图 4-18 所示。

Find:	Logic Utilisation	Used	Available	Utilisation(%)
1	APM	0	240	0
2	FF	8495	133200	7
3	LUT	7507	66600	12
4	Distributed RAM	604	19900	4
5	DRAM	55	155	36
6	IO	94	285	33
7	BCKE	2	24	9
8	SCANCHAIN	1	1	100
9	USCM	9	32	29
10	CCS	1	1	100
11	ADC	0	1	0
12	DORPHY	5	24	21
13	HSSTLP	0.5	1	50
14	GPFL	3	6	50
15	PPLL	2	6	34
16	DORPHY_CPD	1	12	9
17	BCKE	0	96	0
18	IOCKE	0	24	0

图 4-18 FPGA 资源占用原图

通用逻辑（LUT 12%、FF 7%）占用处于低位，表明 RTL 规模与器件容量相称；片上存储（DRAM 36%）、高速收发（HSSTLP 50%）与时钟 PLL（GPFL 50%、PPLL 34%）的占用水平均符合各 IP 核的典型配置，且仍保留足够余量支撑当前 1280×720 RGB565 配置的量产与联调。IO 管脚分配与外设接口一一对应，无冗余浪费或管脚不足现象。报告表明器件选型适当、资源分配均衡、余量充足。

第五章 测试报告

本章给出面向评委的完整验收流程：离线精度 → 板端功能 → PCIe 实时 → FPGA 仿真 → 静态时序与资源。所有精度/延迟指标与快速预览口径一致。测试环境包括：RK3568 开发板、MES2L100H FPGA 板、HDMI 摄像头，以及 CCPD 分层抽样与自采难例共 500 张五类场景验收测试集（晴天/阴天/夜间/多牌/斜拍各 100 张）。



图 6-1 测试与验证总流程

5.1 测试流程与环境

测试分五个阶段串联执行，每阶段均有明确通过准则与记录文件。离线精度阶段使用固定测试集，避免随机抽样带来的口径波动；实时阶段在 PCIe 1280×720 视频流下连续运行不少于 10 分钟，验证稳定性。

阶段	目的	工具/环境	通过准则
A 离线精度	mAP/识别率	Qt 批量 + CLI	mAP≥98%，识别 ≥98%
B 板端功能	单图/多图推理	RK3568 + RKNN	框选号牌正确
C PCIe 实时	延迟与稳定性	FPGA+PCIe+Qt	process_wall≈65ms
D 仿真	采集/缓存/DMA	Modelsim/工具链	波形与协议符合
E 时序资源	可实现性	PDS STA	WNS>0, LUT<20%

环境项	配置
RK3568	Debian + RKNN Runtime 1.5
FPGA	MES2L100H, PCIe Gen2 x2
视频	1280×720 RGB565, stride 4096
测试集	CCPD 分层 + 自采共 500 张

5.2 离线精度测试

验收测试集：CCPD 分层抽样 + 自采难例共 500 张，五类场景各 100 张（晴天/阴天/夜间/多牌/斜拍）；全文精度指标（含快速预览与表 3-2）均以此 500 张为准。评测输出画框结果与 summary.csv。检测端以 mAP@0.5 评估，识别端以整牌字符串准确率评估，同时统计颜色分类准确率。所有测试均在板端 INT8 RKNN 上执行，与实际部署环境一致。

场景	样本数	mAP@0.5	识别准确率
晴天	100	99.1%	99.0%
阴天	100	98.6%	98.4%
夜间	100	97.2%	97.0%
多牌	100	98.0%	97.9%
斜拍	100	97.8%	97.6%
合计/平均	500	98.2%	98.1%

夜间与斜拍场景精度略低于晴天，但均超过 97%，满足快速预览声明。多牌场景验证了双车牌同帧检测与识别能力，与 process_wall ≈ 65 ms 指标相符。

5.3 板端功能与 PCIe 实时测试

功能测试：单图、批量目录、PCIe 实时流三种模式下，检测框、号牌字符串、颜色类别均正常输出；双车牌场景可同帧识别。实时性：连续运行 10 分钟，无

帧丢失/驱动崩溃；process_wall 平均 65 ms，整帧周期平均 98 ms。[latency] 日志记录各阶段耗时，可复现于评审材料。

项目	结果	说明
pcie_dma	27~28 ms	1280×720 RGB565
process_wall	≈ 65 ms	检测+识别+绘制
整帧周期	≈ 98 ms	约 10.2 fps
长稳定性	通过	10 min 连续流

测试项	方法	结论
单图推理	Qt 打开图片	通过
批量评测	CLI + summary.csv	通过
PCIe 实时	HDMI 摄像头流	通过
双车牌	同帧多框	通过

第六章 核心代码注释

本章节摘录系统中具有代表性的核心代码片段，并对关键设计意图进行注释说明。完整源码详见随附“技术数据总压缩包”（含 Verilog 工程、C++ 工程、模型文件及编译说明）。FPGA 端使用紫光同创 Pango Design Suite (PDS) 2022.2 综合实现，RK 端使用 GCC 10.2.1 + CMake 3.18 构建。

6.1 FPGA 端核心代码 (Verilog)

Sobel 边缘检测模块

Sobel 模块采用 3×3 滑动窗口结构，输入为 8 bit 灰度像素流，输出为饱和到 8 bit 的边缘强度。关键代码如下：

```
// sobel_3x3.v - Sobel 边缘检测模块（节选）
module sobel_3x3 #(
    parameter DW = 8                // 像素位宽
)(
    input          clk,
    input          rst_n,
    input          din_valid,        // 输入有效
    input [DW-1:0] din,              // 8 bit 像素
    input          hsync_in,
    input          vsync_in,
    output reg     dout_valid,        // 输出有效
    output reg [DW-1:0] dout,         // 边缘强度（饱和到 8 bit）
    output reg     hsync_out,
    output reg     vsync_out
);

// =====
// 行缓冲区：使用 2 行 BRAM 配合当前行实现 3x3 窗口
// =====
wire [DW-1:0] line0, line1;
line_buffer u_lbuf (
    .clk      (clk),
    .din      (din),
    .din_valid (din_valid),
```



```

        .line0_out (line0),    // 上一行
        .line1_out (line1)    // 上上一行
    );

// =====
// 3x3 窗口寄存器（自动构成 9 个像素的滑动窗口）
// =====
reg [DW-1:0] p00, p01, p02;
reg [DW-1:0] p10, p11, p12;
reg [DW-1:0] p20, p21, p22;
always @(posedge clk) begin
    if (din_valid) begin
        {p02, p01, p00} <= {line1, p02, p01};
        {p12, p11, p10} <= {line0, p12, p11};
        {p22, p21, p20} <= {din,   p22, p21};
    end
end

// =====
// Sobel 卷积：水平方向 Gx + 垂直方向 Gy
// 使用有符号扩展防止溢出
// =====
wire signed [DW+2:0] gx = (p02 + (p12<<1) + p22) - (p00 + (p10<<1) + p20);
wire signed [DW+2:0] gy = (p20 + (p21<<1) + p22) - (p00 + (p01<<1) + p02);

// 取绝对值并相加
wire [DW+2:0] abs_gx = gx[DW+2] ? -gx : gx;
wire [DW+2:0] abs_gy = gy[DW+2] ? -gy : gy;
wire [DW+3:0] mag     = abs_gx + abs_gy;

// 饱和到 8 bit
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        dout      <= 'd0;
        dout_valid <= 1'b0;
    end else begin
        dout      <= (mag > 'd255) ? 8'hFF : mag[DW-1:0];
        dout_valid <= din_valid;
    end
end

```

```

        end
    end
endmodule

```

【设计要点说明】

- 使用 BRAM 实现的行缓冲器替代寄存器阵列，每像素只占用 2 个存储位，资源占用较纯寄存器实现降低约 70 %。
- 为防止 Gx/Gy 中间结果溢出，运算过程统一使用 (DW+3) 位有符号扩展。
- 最终输出做 8 bit 饱和限幅，避免回卷 (wrap-around)。

PCIe DMA 控制器

PCIe DMA 控制器采用“主从 + 描述符”模式，下面是描述符 FIFO 与 TLP 发送状态机的关键片段：

```

// pcie_dma_tx.v - PCIe DMA 发送控制器（节选）
module pcie_dma_tx (
    input          clk_pcie,          // 125 MHz PCIe 用户时钟
    input          rst_n,
    // 描述符输入接口（从寄存器配置）
    input          desc_valid,
    input [63:0]   desc_dst_addr,     // RK3568 主存目的地址（DMA 物理地址）
    input [31:0]   desc_length,       // 长度（字节）
    output         desc_ready,
    // PCIe TX TLP 接口
    output reg [127:0] tx_tdata,
    output reg [15:0] tx_tkeep,
    output reg     tx_tvalid,
    output reg     tx_tlast,
    input          tx_tready,
    // 数据源接口
    input [127:0]   src_tdata,
    input          src_tvalid,
    output         src_tready
);
    // =====
    // 状态机：IDLE -> HEADER -> PAYLOAD -> DONE

```

```

// 每次发送一个 256 B 的 TLP (MaxPayload = 256)
// =====
localparam S_IDLE    = 3'd0;
localparam S_HEADER  = 3'd1;
localparam S_PAYLOAD = 3'd2;
localparam S_DONE    = 3'd3;
reg [2:0] state;
reg [31:0] bytes_left;    // 本次描述符剩余字节
reg [31:0] payload_cnt;   // 当前 TLP 已发字节数
localparam MAX_PAYLOAD = 256;

always @(posedge clk_pcie or negedge rst_n) begin
    if (!rst_n) begin
        state      <= S_IDLE;
        bytes_left <= 0;
        payload_cnt <= 0;
    end else begin
        case (state)
            // 等待新描述符
            S_IDLE: if (desc_valid) begin
                bytes_left <= desc_length;
                state      <= S_HEADER;
            end
            // 发送 PCIe Memory Write TLP 头 (4 DW, 128 bit 对齐)
            S_HEADER: if (tx_tready) begin
                tx_tdata    <= build_mwr_header(desc_dst_addr, MAX_PAYLOAD);
                tx_tvalid   <= 1'b1;
                state       <= S_PAYLOAD;
                payload_cnt <= 0;
            end
            // 发送 payload
            S_PAYLOAD: if (tx_tready && src_tvalid) begin
                tx_tdata    <= src_tdata;
                tx_tlast    <= (payload_cnt + 16 >= MAX_PAYLOAD);
                payload_cnt <= payload_cnt + 16;
                if (payload_cnt + 16 >= MAX_PAYLOAD) begin
                    bytes_left <= bytes_left - MAX_PAYLOAD;
                    state <= (bytes_left <= MAX_PAYLOAD) ? S_DONE : S_HEADER;
                end
            end
        endcase
    end
end

```

```

        end
    end
    // 整次描述符完成，触发 MSI 中断
    S_DONE: begin
        msi_req <= 1'b1;
        state    <= S_IDLE;
    end
endcase
end
end
endmodule

```

【设计要点说明】

- MaxPayload 设为 256 B，与 RK3568 PCIe RC 协商后的实际值匹配，过大或过小均会损失带宽。
- 状态机为四态精简结构，所有跳转都基于 tx_tready 反压信号，避免数据丢失。
- 每次描述符完成产生一次 MSI 中断，RK 端可在中断处理中切换缓冲区，实现双缓冲零拷贝。

6.2 RK 端核心代码（C++）

R0I 精细化定位与几何校正

RK 端使用 OpenCV，关键代码节选如下：

```

// roi_refine.cpp - ROI 精细化定位（节选）
#include <opencv2/opencv.hpp>

// 输入：FPGA 输出的粗 ROI（原图坐标系）；输出：校正后的车牌图
cv::Mat refineROI(const cv::Mat &fullImg, const cv::Rect &roi) {
    // -----
    // 1. 在原图上扩展 ROI（避免边缘截断），并裁剪
    // -----
    int pad = 8;
    cv::Rect rExp = roi & cv::Rect(0, 0, fullImg.cols, fullImg.rows);
    rExp.x = std::max(0, rExp.x - pad);

```

```

rExp.y = std::max(0, rExp.y - pad);
rExp.width = std::min(fullImg.cols - rExp.x, rExp.width + 2 * pad);
rExp.height = std::min(fullImg.rows - rExp.y, rExp.height + 2 * pad);
cv::Mat patch = fullImg(rExp).clone();

// -----
// 2. 灰度化 + Canny 边缘检测, 准备做霍夫直线
// -----
cv::Mat gray, edges;
cv::cvtColor(patch, gray, cv::COLOR_BGR2GRAY);
cv::Canny(gray, edges, 80, 200);

// -----
// 3. 概率霍夫变换提取主导直线, 估计倾角
// -----
std::vector<cv::Vec4i> lines;
cv::HoughLinesP(edges, lines, 1, CV_PI / 180,
                 50, patch.cols * 0.4, 10);
double angle = 0.0;
int validCnt = 0;
for (const auto &l : lines) {
    double dx = l[2] - l[0];
    double dy = l[3] - l[1];
    double a = std::atan2(dy, dx) * 180.0 / CV_PI;
    if (std::abs(a) < 30.0) { // 仅采用近似水平的直线
        angle += a;
        validCnt++;
    }
}
if (validCnt > 0) angle /= validCnt;

// -----
// 4. 仿射旋转校正
// -----
cv::Point2f center(patch.cols / 2.0f, patch.rows / 2.0f);
cv::Mat rot = cv::getRotationMatrix2D(center, angle, 1.0);
cv::Mat rectified;
cv::warpAffine(patch, rectified, rot, patch.size(),

```

```

        cv::INTER_LINEAR, cv::BORDER_REPLICATE);

// -----
// 5. 二次定位：投影法精确收紧到字符区域
// -----
cv::Mat bin;
cv::cvtColor(rectified, gray, cv::COLOR_BGR2GRAY);
cv::adaptiveThreshold(gray, bin, 255,
                      cv::ADAPTIVE_THRESH_GAUSSIAN_C,
                      cv::THRESH_BINARY_INV, 25, 10);
cv::Rect tight = projectionTighten(bin); // 自定义投影函数
cv::Mat plate = rectified(tight).clone();

// -----
// 6. resize 到 LPRNet 需求的 94x24 输入大小
// -----
cv::Mat plateResized;
cv::resize(plate, plateResized, cv::Size(94, 24));
return plateResized;
}

```

LPR 模型推理调用

使用 RKNN API 加载 INT8 量化后的 .rknn 模型，并完成推理：

```

// lpr_infer.cpp - 调用 NPU 进行车牌字符识别（节选）
#include "rknn_api.h"
#include <opencv2/opencv.hpp>

class LPRInference {
public:
    int init(const std::string &modelPath) {
        // 1. 读取 .rknn 模型
        std::ifstream f(modelPath, std::ios::binary);
        std::vector<char> buf((std::istreambuf_iterator<char>(f)),
                              std::istreambuf_iterator<char>());

        // 2. 初始化 RKNN context
        int ret = rknn_init(&ctx_, buf.data(), buf.size(), 0, nullptr);
        if (ret < 0) return ret;
    }
};

```

```

// 3. 查询输入/输出张量属性
rknn_query(ctx_, RKNN_QUERY_IN_OUT_NUM, &io_num_, sizeof(io_num_));
return 0;
}

// 输入: 94x24 BGR 车牌图, 输出: 字符串
std::string infer(const cv::Mat &plate) {
    // 1. 准备输入: HWC -> CHW, 归一化到 [-1, 1]
    rknn_input inputs[1];
    memset(inputs, 0, sizeof(inputs));
    inputs[0].index = 0;
    inputs[0].type = RKNN_TENSOR_UINT8;           // INT8 量化模型直接接收
uint8
    inputs[0].fmt = RKNN_TENSOR_NHWC;
    inputs[0].size = plate.total() * plate.channels();
    inputs[0].buf = plate.data;
    rknn_inputs_set(ctx_, 1, inputs);

    // 2. 同步推理
    rknn_run(ctx_, nullptr);

    // 3. 取出输出 (shape: [1, 18, 68])
    rknn_output outputs[1];
    memset(outputs, 0, sizeof(outputs));
    outputs[0].want_float = 1;
    rknn_outputs_get(ctx_, 1, outputs, nullptr);
    float *logits = (float *)outputs[0].buf;

    // 4. CTC 贪心解码
    std::string result;
    int prev = -1;
    const int T = 18, C = 68;
    for (int t = 0; t < T; ++t) {
        int cls = std::max_element(logits + t * C,
                                   logits + (t + 1) * C) - (logits + t * C);
        if (cls != BLANK_IDX && cls != prev) {
            result += CHAR_TABLE[cls];
        }
    }
}

```



```

        prev = cls;
    }
    rknn_outputs_release(ctx_, 1, outputs);
    return result;
}

private:
    rknn_context ctx_;
    rknn_input_output_num io_num_;
};

```

【设计要点说明】

INT8 量化模型可直接接收 uint8 原始像素，无需在 CPU 上做 float 归一化，节省约 1.2 ms 数据预处理时间。

CTC 解码逻辑简洁，时间步 $T = 18$ ，类别数 $C = 68$ ，单帧解码 $< 0.05 \text{ ms}$ 。

整个推理调用线程化封装，可并行处理多 ROI；